

Hieronym: Leveraging Hierarchical Multi-Source Information for Function Renaming in Stripped Binary

Xiaoling Zhang
Zhongguancun Laboratory
Beijing, China
zhangxl@zgclab.edu.cn

Jian Sun
Zhongguancun Laboratory
Beijing, China
sunjian@zgclab.edu.cn

Dawei Wang*
Zhongguancun Laboratory
Beijing, China
wangdw@zgclab.edu.cn

Chongyu Wang
Zhongguancun Laboratory
Beijing, China
wangchy@zgclab.edu.cn

Li Chen*
Zhongguancun Laboratory
Beijing, China
lichen@zgclab.edu.cn

Zhaoteng Yan
Zhongguancun Laboratory
Beijing, China
yanzt@zgclab.edu.cn

Peipei Liu
Zhongguancun Laboratory
Beijing, China
liupp@zgclab.edu.cn

Lixiao Zhang
Zhongguancun Laboratory
Beijing, China
Harbin Institute of Technology
Harbin, China
zhanglx2025@zgclab.edu.cn

Dan Li
Tsinghua University
Beijing, China
tolidan@tsinghua.edu.cn

Abstract

Function renaming in stripped binaries can substantially assist reverse engineers by improving code readability, yet it is a challenging task. The difficulty stems from the need to accurately capture function semantics from low-level binary code across diverse instruction sets, architectures, and compiler optimizations, and to express these semantics in concise, human-readable names. Existing approaches either inadequately capture comprehensive function semantics or exhibit limited generalization to previously unseen binaries. In this paper, we present HIERONYM, a generative large language model (LLM)-based framework for stripped binary function renaming. HIERONYM adopts a hierarchical summarization-driven domain adaptation strategy and integrates multi-source information—including global binary context, local calling context, and intrinsic function semantics—to enhance the LLM’s understanding of binary code. To enable systematic evaluation, we further propose a dual-layer evaluation framework that incorporates both token-level and whole-name-level metrics. We evaluate HIERONYM on binary functions compiled with four compiler optimization levels (O0–O3) for four architectures (x64, x86, ARM, and MIPS). Experimental results demonstrate that HIERONYM significantly outperforms state-of-the-art methods, achieving token-level improvements of 50.12% in precision, 41.75% in recall, and 45.10% in F1-score, as well as a 79.94% improvement in name-level accuracy, while also exhibiting

strong generalization capability. Moreover, experiments on real-world malware samples further validate the practical effectiveness of HIERONYM in security-critical scenarios.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Security and privacy** → **Software reverse engineering**.

Keywords

Reverse Engineering, Large Language Models, Binary Function Renaming, Program Comprehension

ACM Reference Format:

Xiaoling Zhang, Jian Sun, Dawei Wang, Chongyu Wang, Li Chen, Zhaoteng Yan, Peipei Liu, Lixiao Zhang, and Dan Li. 2026. Hieronym: Leveraging Hierarchical Multi-Source Information for Function Renaming in Stripped Binary. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 20 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Reverse engineering is a fundamental technique in security analysis, supporting tasks such as vulnerability discovery [63, 69], malware analysis [2, 71], and security auditing [23]. In practice, analysts frequently encounter stripped binaries lacking high-level information, including function and variable names, structure details and optimization information. This loss obscures program intent and hinders analysis efficiency. Among the missing information, function names are particularly critical, as they provide concise abstractions of program functionality and serve as key cognitive anchors during program comprehension [10, 61]. However, widely used decompilation tools such as IDA Pro and Ghidra typically assign address-based placeholder names (such as FUN_00129f48) that convey little information. Accurate function naming still depends heavily on manual expert analysis [72], rendering the process

*Corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/06
<https://doi.org/XXXXXXX.XXXXXXX>

time-consuming and labor-intensive. Consequently, automatic function renaming for stripped binaries has emerged as an important research direction.

However, automatic function renaming is a challenging task, as it requires both a comprehensive understanding of binary function semantics and effective abstraction of such semantics into concise and meaningful natural-language keywords. Recent machine learning (ML)-based approaches have shown promise [20, 27, 38, 49, 50, 73]. These methods typically employ weak base models, such as LSTM [77] in NFRE [20] and BERT [14] in SymLM [38], to understand function semantics and adopt a classification-based paradigm that maps function semantics to a predefined, closed vocabulary. These models often struggle to capture function semantics robustly, and exhibit limited ability to abstract function semantics into natural-language keywords. While they may perform well on training data, they often lack sufficient generalization to unseen binaries. Generative large language models (LLMs) have recently demonstrated strong generalization and reasoning capabilities, achieving impressive results in code understanding and generation. These advances have spurred growing interest in applying LLMs to binary function renaming. However, despite achieving state-of-the-art (SOTA) performance, existing LLM-based methods still fall short of practical expectations: even the best-performing approach achieves only a 0.277 F1-score. This discrepancy raises a critical question: **why have existing methods not delivered the performance we would expect from LLMs in this setting?** We analyze this gap and identify the following challenges.

C1: Flawed Evaluation Metrics. Prior evaluation metrics for function renaming are flawed, as they sometimes require models to reproduce fine-grained details unavailable from binary function alone. For example, the prefix in `jitterc_get_lineno` encodes project-specific context unrecoverable from the binary functions alone. Penalizing the prediction of `get_current_line_number` is therefore unjustified. Moreover, most evaluations rely on token-level metrics to mitigate the out-of-vocabulary (OOV) issue, but this choice is fragile. Erroneous tokenization can drastically distort evaluation metrics. For instance, SymGen [35] tokenizes `c_isascii_string` into `{c, isa, sc, ii, string}`, reducing its F1-score from 0.857 to 0.25. Since a sound evaluation is essential for revealing the true limitations of existing methods and supporting a faithful interpretation of their performance, how to properly evaluate the performance of the function renaming task is a non-trivial challenge.

C2: Semantic Sparsity of Stripped Binaries. Motivated by C1, we observe that stripped binary functions contain only limited semantic information, which often makes accurate name recovery unattainable. For example, the `jitterc`-specific identifier discussed above is essentially impossible to recover from a single stripped binary function. Although prior works, such as SymLM [38], XFL [50], and BLens [4], attempt to incorporate additional context, such as calling context, this information is often still insufficient to compensate for the loss of semantics in stripped binaries, especially for context-specific identifiers. Moreover, the contextual representations used by these methods are sensitive to architectural and optimization variations, which limits their robustness. Therefore, a key challenge is how to model essential, task-relevant context from information-limited binaries while remaining robust to architecture- and optimization-induced changes.

C3: Knowledge Gaps Between Natural Language and Binary Code Semantics in LLMs. Although LLMs are strong at understanding and generating source code, their training corpora are dominated by natural language and include relatively little code, which leaves them with limited ability to understand stripped binary [51, 59]. This mismatch creates a substantial gap between the knowledge encoded in general-purpose LLMs and the understanding required for binary analysis. A straightforward approach to mitigating this gap is domain adaptation. Accordingly, function summarization-based domain adaptation strategies have been explored and shown some effectiveness in transferring LLMs to the binary domain [35]. However, prior studies indicate that function-level summaries alone are insufficient to capture deep and comprehensive function semantics [24, 31–33, 47]. Consequently, effectively bridging the knowledge gap between general-purpose LLMs and the binary domain remains an urgent open problem.

In this paper, we present HIERONYM, a novel generative framework for function renaming in stripped binaries. It is designed to learn hierarchical binary function semantics and to comprehensively model binary functions by integrating multi-source information. Leveraging generative LLMs, HIERONYM abstracts binary semantics into natural-language keywords. To ensure comprehensive and reliable evaluation (C1), we adopt a dual-layer evaluation framework that assesses performance at both the token and name levels. We propose an enhanced token-level evaluation with a hybrid tokenization mechanism to produce semantically meaningful tokens to mitigate tokenization errors, along with heuristic rules to measure token-level semantic relatedness. Our evaluation framework is further completed by a name-level evaluation using an LLM-as-a-judge approach to assess whole-name semantic equivalence. To comprehensively model function semantics (C2), HIERONYM integrates three complementary information sources: global binary context, local calling context, and the target function itself. Global and local contexts provide essential task-relevant information, which we represent using textual summaries. Our representation offers a robust abstraction that is resilient to code variations introduced by different compilation. To bridge the knowledge gap between general-purpose LLMs and the binary domain (C3), we introduce hierarchical function summarization as a domain adaptation strategy, enabling multi-view semantic understanding of functions.

Our comprehensive evaluation demonstrates that HIERONYM exhibits exceptional generalizability and superior performance compared to SOTA function renaming methods, including SymLM [38], XFL [50], BLens [4], and SymGen [35], across four instruction set architectures (x64, x86, ARM, and MIPS) and four optimization levels (O0–O3). HIERONYM achieves substantial gains in token-level precision, recall, and F1-score of 50.12%, 41.75%, and 45.10%, respectively, as well as a 79.94% improvement in name-level accuracy over the best prior method. Leveraging the multi-source information in our approach, HIERONYM even successfully recovers the `jitterc`-specific identifier discussed above. Ablation studies further validate the effectiveness of HIERONYM’s design. Hierarchical function summarization based domain adaptation improves F1-score by 18.83%, while incorporating global binary context and local calling context yields additional gains of 27.63% and 13.81%, respectively. These results highlight the contributions of each component and underscore

the novelty and effectiveness of the proposed framework. Finally, experiments on real-world malware binaries confirm HIERONYM’s practical utility for security analysis.

The contributions of this paper can be summarized as follows:

- We propose a dual-layer evaluation framework that evaluates performance at both the token-level and the name-level, enabling a reliable evaluation of function renaming methods.
- We propose HIERONYM, a novel generative approach for function renaming in stripped binaries that leverages multi-source information. By integrating global binary context, local calling context, and target function, HIERONYM enables LLMs to learn comprehensive function semantics, resulting in improved performance and generalizability.
- We introduce a hierarchical function summarization-based domain adaptation strategy that effectively adapts pre-trained generative LLMs to binary semantic modeling.
- Extensive experiments show that HIERONYM consistently outperforms SOTA methods in function renaming. The results further highlight its strong generalizability, the effectiveness of individual components, and its practical applicability. We make our code and dataset publicly available at: <https://github.com/NASPTU/Hieronym>.

2 Background and Motivation

2.1 Problem Definition

Function renaming in stripped binaries requires both accurate reverse engineering of binary semantics and the generation of names that faithfully reflect those semantics. Decoder-only LLMs, based on the Transformer decoder architecture [51, 59], have demonstrated strong capabilities in text understanding and generation, particularly exhibiting impressive generalization in zero- and few-shot settings [6]. In contrast to existing ML-based approaches [20, 38, 73], which formulate function renaming as a classification task over a closed vocabulary, we follow SymGen [35] and model function renaming as an autoregressive generation task using LLMs. Formally, given a binary function $\mathcal{F}$, represented as a sequence of binary code tokens $\mathcal{F} = \{t_1, t_2, \dots, t_n\}$, a function renaming model $\mathcal{M}$ generates a sequence of natural-language tokens $\mathcal{W} = \{w_1, w_2, \dots, w_m\}$, i.e., $\mathcal{M} : \mathcal{F} \rightarrow \mathcal{W}$. Under the autoregressive paradigm, the generation probability of $\mathcal{W}$ is factorized as

$$p(\mathcal{W}) = \prod_{j=1}^m p(w_j \mid w_1, \dots, w_{j-1}, \mathcal{F}), \quad (1)$$

where each token w_j is conditioned on all previously generated tokens and the binary function semantics.

2.2 Existing Techniques and Our Motivations

Although many studies have explored function renaming in stripped binaries, prior efforts have not adequately addressed the challenges outlined above, which directly motivates our proposal of HIERONYM.

Existing methods primarily evaluate function-renaming performance at the token level, yet the reliability of such evaluation heavily depends on the underlying tokenization strategy, which directly affects subsequent semantic alignment. Many methods—including XFL [50], BLens [4], SymLM [38], and SymGen [35]—rely on rule-based tokenization, which often introduces segmentation bias and

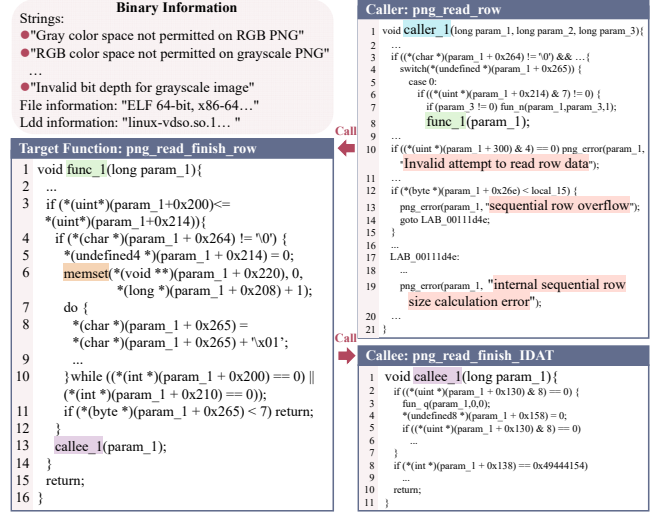


Figure 1: An Example Illustrating the Importance of Task-Related Information in Function Naming

degrades evaluation metrics. For example, SymLM incorrectly splits `test_nofork_sideeffects` into `{t, est, no, for, k, side, eff, ect, s}`, hindering correct alignment with the ground-truth tokens. Semantic alignment based on tokenization results also remains limited. XFL and BLens depend on general English synonym sets that fail to capture domain-specific morphological variations, while the CodeWordNet [38] models used by SymLM and SymGen are trained on small corpora with insufficient coverage of OOV tokens. As a result, existing token-level evaluation strategies suffer from unreliable tokenization and weak semantic alignment, and the absence of name-level evaluation further prevents current methods from systematically addressing C1.

Grasping task-relevant contextual information is essential for modeling function semantics and improving function-renaming performance. Figure 1 illustrates this through a real-world example of a stripped binary, showing the decompiled code of a target function together with its callers, callees, and domain-related information extracted from binary (simplified for clarity). The example involves three functions—`func_1`, `caller_1`, and `callee_1`—whose ground-truth names are `png_read_finish_row`, `png_read_row`, and `png_read_finish_IDAT`, respectively. Although decompiled code exposes richer semantics than raw binaries, the target function alone is insufficient for accurate name inference. As shown in Figure 1, the semantics of `png_read_finish_row` are tightly coupled with both its caller and callee. Prior works have attempted to incorporate calling context in different ways. SymLM models calling context using assembly-level representations, while XFL and BLens rely on handcrafted features to capture calling relationships. However, these methods are sensitive to architecture and optimization settings, causing unstable semantics (e.g., SymLM reports a 60.2% F1-score drop from O3 to O0). Domain-related information (e.g., the keyword “PNG”) provides essential task-relevant cues unavailable from the local function alone. Despite its importance, existing methods do not consider such domain-related information. The

LLM-based method SymGen ignores both domain information and calling context. Existing methods lack effective mechanisms for leveraging task-relevant information and struggle to address C2.

Many existing methods, such as SymLM, BLens, and XFL, are built on traditional ML frameworks and lack intrinsic mechanisms to address C3. SymGen introduces function-level summarization to equip LLMs with binary code knowledge. However, prior studies show that function-level summaries alone are insufficient to capture fine-grained implementation details. In particular, statement-level summarization has been shown to effectively complement function-level semantics by modeling the distinct logical operations within a function [24, 33]. These two levels of abstraction are inherently complementary: statement-level summaries capture concrete implementation logic, while function-level summaries convey overall functionality. Together, they provide a more complete and nuanced understanding of code structure and semantics [24, 31]. As a result, SymGen remains limited in modeling fine-grained binary semantics and does not fully address C3.

3 Evaluation Framework

To comprehensively evaluate function renaming methods, we propose a dual-level evaluation framework: (1) **token-level evaluation**, which is widely used in prior work to mitigate OOV issue; and (2) **name-level evaluation**, which measures the correctness of generated function names from a holistic semantic perspective. By combining these two complementary levels, our framework enables a systematic evaluation of both fine-grained semantic recognition and overall semantic expressiveness.

3.1 Token-Level Evaluation

Token-level evaluation typically consists of two stages: (1) function-name tokenization, which segments a name into meaningful tokens, and (2) token semantic alignment, which determines semantic relationships among tokens. In the tokenization stage, we adopt a hybrid strategy that combines rule-based heuristics with LLM assistance. In the subsequent alignment stage, heuristic rules are applied to identify semantic associations between tokens.

Name Tokenization. Function names are typically multi-word natural-language expressions that include abbreviations, domain-specific jargon, and language-dependent conventions [29, 53]. Unlike natural language text with explicit word boundaries, code supports diverse naming styles. Although capitalization, underscores, or digits are commonly used as delimiters, they are often omitted, resulting in unsegmented character sequences with ambiguous word boundaries. For function names that follow conventional naming styles—such as camelCase, underscore-separated, or numeric-delimited forms—we apply a rule-based tokenization strategy. For unconventional names, rule-based methods used in prior work [35, 38] tend to over-segment names, breaking coherent semantic tokens and thereby undermining token-level evaluation. To address this limitation, we introduce an LLM-based few-shot tokenization approach that leverages contextual understanding to achieve accurate segmentation of complex function names. The prompt is illustrated in Figure 13 in Appendix C. We provide the

LLM with a small set of diverse in-context examples covering different naming styles and domain-related vocabulary to guide structured parsing. For each function name, we generate three independent tokenization results and aggregate them using a voting-based strategy. The final segmentation is determined as follows:

- A segmentation boundary is retained if it is suggested in at least two of the three results.
- Among the three candidates, the result with the largest intersection with the retained boundary set is selected, with any redundant or overlapping labels removed.

By integrating rule-based heuristics with LLM-driven tokenization, our framework achieves robust and effective segmentation of function names, providing a reliable foundation for subsequent token-level semantic analysis.

Token Semantic Alignment. Function names exhibit substantial lexical variability due to developer preferences, including morphological variants, abbreviations, and misspellings [29, 36, 38, 53]. Ignoring such relationships significantly biases evaluation, and manual curation does not scale to large vocabularies. To address this issue, we build on prior work [20, 73] and propose an automated method for identifying semantically related tokens. We categorize token relationships into four types and handle each as follows:

- For morphological variants, we apply lemmatization to determine whether two tokens share the same lexical root.
- For abbreviation, we use Smith-Waterman algorithm [55] to compute token similarity. If the relative similarity exceeds 0.65 and both tokens are longer than two characters, they are considered to form an abbreviation-full-form relationship.
- For misspellings, we propose a Damerau-Levenshtein-based measure [3] to quantify token similarity, pairs exceeding a threshold of 0.75 are classified as spelling variants.
- For synonyms, we use Qwen3-Embedding [75] to mitigate OOV issues, trained on large-scale corpora, pairs with similarity scores above 0.8 are treated as synonyms.

By incorporating multi-dimensional association mechanism, our evaluation framework can identify semantically equivalent tokens, improving evaluation robustness in the presence of lexical diversity. The thresholds used in this mechanism are determined through empirical analysis and validated via experimental evaluation.

Token-Level Metrics. To evaluate our model and existing methods, we adopt the standard token-level precision, recall, and F1-score metrics, following common practice in prior studies [4, 35, 38, 73]. These metrics quantify the overlap between tokens in the predicted and ground-truth function names. We define *true positives* (TP) as the number of correctly predicted tokens, including both exact matches and those considered equivalent under semantic alignment. *False positives* (FP) are the number of predicted tokens that do not appear in ground-truth tokens, while *false negatives* (FN) are the number of ground-truth tokens that are not predicted. Formally, given a predicted name $\hat{W} : \{\hat{w}_1, \dots, \hat{w}_j\}$ and the ground truth $W : \{w_1, \dots, w_m\}$, where $w_t(\hat{w}_t)$ denotes a token in the name, and α is an indicator function such that $\alpha\{True\} = 1$ and $\alpha\{False\} = 0$, we compute:

$$TP = \sum_{t=1}^j \alpha(\hat{w}_t \in W) \quad FP = \sum_{t=1}^j \alpha(\hat{w}_t \notin W) \quad FN = \sum_{t=1}^m \alpha(w_t \notin \hat{W})$$

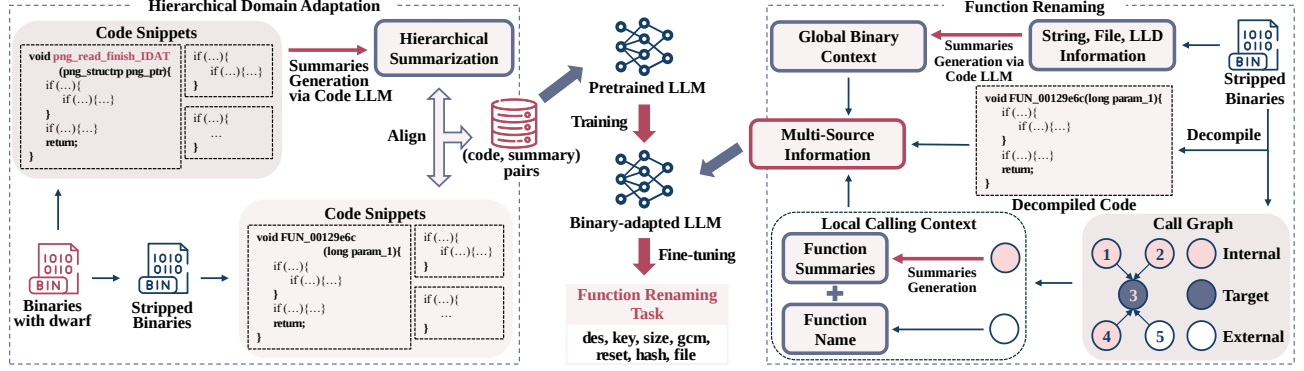


Figure 2: The Overall Workflow of HIERONYM

Precision (P), recall (R), and F1-score (F1) are then computed as:

$$P = \frac{TP}{TP + FP} \quad R = \frac{TP}{TP + FN} \quad F1 = \frac{2 \times P \times R}{P + R}$$

3.2 Name-Level Evaluation

Relying solely on token-level evaluation is insufficient for evaluating semantic consistency between predicted and ground-truth function names. Prior studies show that even for identical program semantics, developers choose the same identifier with a probability of only 6.9% [18]. Function names may be lexically dissimilar yet semantically equivalent, causing token-level metrics (e.g., F1-score) to underestimate model performance. We introduce a name-level evaluation that assesses semantic consistency from a holistic perspective to address this limitation. Recent studies indicate that LLMs used as evaluators align more closely with human judgment [56, 70]. Inspired by this finding, we adopt an LLM-assisted evaluation strategy and employ GPT-4o [45] as a semantic judge. For each binary function, we query GPT-4o with the global binary information, its decompiled code, and a pair consisting of the ground-truth name and the predicted name. GPT-4o is instructed to determine whether the predicted name is semantically consistent with the ground-truth name and to output a binary decision (“yes” or “no”). The detailed prompt is provided in Figure 14 of Appendix C.

To mitigate potential hallucinations or subjectivity in LLM-based evaluation, we further incorporate an embedding-based semantic similarity check. We compute the cosine similarity between the predicted and ground-truth names using the Qwen3-Embedding model and apply a threshold of 0.7 determined through empirical validation. A prediction is considered semantically correct only if both the GPT-4o judgment and the embedding-based similarity criterion indicate agreement.

Name-Level Metric. We evaluate name-level performance using accuracy, defined as the proportion of predicted function names judged correct under the above criteria. Due to the scale of the dataset and the cost of LLM-based evaluation, we follow common practice [37, 70] and randomly sample 200 test functions for each architecture–optimization pair, yielding a total of 3,200 functions for name-level evaluation. This sampling strategy balances statistical reliability with computational feasibility.

4 Approach

4.1 Overview

Reverse-engineering practice indicates that analysts typically develop a global overview of a binary before examining functions in detail [43, 61], such as its domain or functional scope. Motivated by this coarse-to-fine reasoning process and by the challenges discussed above, we propose HIERONYM, a function renaming framework that leverages hierarchical code comprehension and multi-source information to robustly handle binaries compiled under diverse optimization levels. HIERONYM consists of three key steps: (I) **Hierarchical Domain Adaptation**, which enhances LLMs’ understanding of binary semantics at multiple granularities; (II) **Multi-Source Information Fusion**, which integrates global binary context, local calling context, and the target function to construct a comprehensive semantic representation; and (III) **Function Renaming Fine-Tuning**, which applies the fused information to the domain-adapted model to enable accurate and generalizable function renaming.

4.2 Hierarchical Domain Adaptation

We build a binary function renaming model upon a pre-trained generative LLM, leveraging its strong semantic modeling capability to capture complex binary function semantics and generalize across diverse binaries. Although general-purpose LLMs are exposed to a limited amount of source code during pre-training (e.g., approximately 5% in the LLaMA dataset [59]), a substantial semantic gap remains between source code and binary code. Source code preserves rich information—such as variable names and types, and structural cues—whereas stripped binaries lose most of these details, rendering direct application of LLMs to such binaries ineffective.

To bridge this gap, we propose a hierarchical summarization–driven domain adaptation strategy, illustrated in Figure 2. We define a code snippet as a function and its constituent statements, where statements correspond to control-flow constructs (e.g., if–else, while, and switch). This strategy employs summaries at two complementary levels to provide a complete and nuanced representation of binary function semantics: function-level summaries capture high-level functional intent, while statement-level summaries model fine-grained implementation logic. Specifically, we extract statements from the abstract syntax tree (AST) and employ an advanced LLM to

generate summaries for both statements and entire functions, forming a hierarchical representation that are used to systematically enhance HIERONYM's understanding of binary semantics.

Statement-Level Data Preparation. Compared with source code, binaries with symbol tables are structurally closer to their stripped counterparts while preserving richer information, such as identifier names and type details. Leveraging this property, we parse decompiled functions from binaries with symbol tables using Tree-sitter [7] to construct ASTs. From the ASTs, we identify nodes corresponding to control-flow constructs—such as `if-else`, `for`, `while`, `do-while`, and `switch`—as candidate statements, and further decompose nested constructs into multiple independent candidate statements. To improve statement quality, we apply a two-stage refinement process. First, we merge overly short fragments (those containing fewer than five lines of meaningful code) to reduce noise. Second, we merge overlapping fragments whose line-count difference is less than eight lines to eliminate redundancy. The resulting set consists of moderate-sized statements, which serve as the fundamental units for subsequent hierarchical summary generation.

Summary Generation via Code LLMs. Building on the success of code-specialized LLMs (such as Qwen3-Coder [58] and Code Llama [52]) in generating high-quality function summaries, we leverage these models to guide binary code analysis. We do not use original source-code comments, as prior studies show that LLM-generated summaries yield significantly better performance than source-code comments for function renaming [35]. Motivated by these findings, we employ Qwen3-Coder to generate summaries of code snippet from decompiled binaries that retain symbol tables. These summaries provide high-quality semantic supervision for adapting HIERONYM to the binary domain. During summary generation, we guide the LLM to focus on stable features (e.g., inputs/outputs, key steps, and logical structure), ensuring closer alignment with the characteristics of stripped decompiled code.

To improve reliability, we employ self-consistency checking [9, 44, 64]: multiple candidate summaries are generated through repeated LLM queries, the model then evaluates their mutual consistency, and the most consistent summary is selected as the final output. Through experimentation with different prompting strategies, we identify an optimal prompt template (shown in Figures 8 and 10 in Appendix A).

Alignment of Stripped Binaries and Summaries. To associate summaries derived from DWARF-enabled binaries with stripped binaries, we perform alignment at both the function and statement levels, constructing (stripped function/statement, summary) pairs for training the binary-adapted LLM, as shown in Figure 2. We first conduct function-level alignment by parsing DWARF information and matching stripped functions to their unstripped counterparts based on function addresses (i.e., start and end addresses). As symbol-table stripping does not modify function address, the address-based matching establishes a reliable correspondence, enabling accurate association of summaries with stripped functions.

We then perform statement-level alignment. The stripping process may alter internal function structure, so the line-number-based matching is unreliable. To address this issue, we propose a hybrid alignment strategy that combines structural and semantic similarity to match statements between stripped and unstripped binaries, making the alignment robust to local variations introduced by stripping.

Specifically, we parse decompiled code using Tree-sitter to identify logical statements (e.g., `if-else`, `for`, `while`). For each statement, we construct its AST and extract type paths from the root to leaf nodes. Structural similarity is measured using the Jaccard similarity over these AST-path sets. We obtain semantic embeddings for each statement using a pre-trained CodeBERT model [19] and compute semantic similarity via cosine similarity. The matching score is defined as a weighted combination of structural and semantic similarities. During alignment, we enforce consistency in statement types and accept a match only if its score exceeds a predefined threshold. This procedure enables mapping of stripped statements to summaries generated from their unstripped counterparts.

Based on the above design, we adopt hierarchical summarization as our domain-adaptation strategy. Specifically, we pair decompiled stripped binary code snippets with the corresponding summaries and further fine-tune HIERONYM to adapt it to binary semantics.

4.3 Multi-Source Information Fusion

To acquire task-relevant semantics, we introduce a semantic fusion module that integrates multi-source information to construct a comprehensive representation of function semantics. Specifically, the module incorporates three complementary sources: global binary context, local calling context, and the target function itself. This section describes the representation of each information.

Global Binary Context. Motivated by reverse engineering practice, we extract domain-relevant information from binaries and denote it as global binary context. String literals—which may remain present even in stripped binaries—serve as a primary source of domain-relevant information. These strings are generally stable across compilation settings and often encode domain- or function-related semantic cues. We extract candidate strings by retaining sequences longer than eight characters and applying heuristic filters to remove meaningless patterns. Specifically, we discard low-quality strings (e.g., low-diversity or non-alphanumeric-dominated patterns) and assign each remaining string a quality score in $[0,1]$ based on alphanumeric ratio, proportion of non-special characters, and penalties for consecutive special characters. Only strings with scores above 0.65 are retained, while system-related strings (e.g., compilation paths) are preserved. From the filtered set, we randomly sample 100 strings and augment them with file-type indicators and dynamic dependency information to construct the final string set. File information provides insights into architecture and build characteristics, while dynamic dependencies that indicate domains (e.g., `libssl/libcurl` suggesting encryption/networking), jointly helping to constrain the binary's domain. However, string content varies substantially across binaries, and naively concatenating all strings can easily exceed the context-length limits of HIERONYM during fine-tuning. To address this issue, we leverage the summarization capability of an LLM to generate a fixed-length textual summary from the final string set. This summary captures high-level domain context, yielding a compact yet semantically rich representation of global binary context, as illustrated in Figure 7 of the Appendix A.

Local Calling Context. We construct local calling context based on the direct call relationships of the target function. Since call-path depth varies significantly across functions, incorporating all indirect relationships would produce excessively large call graphs,

incurring substantial computational and memory overhead during training and inference. To balance expressiveness and efficiency, we follow prior work [8, 15, 38] and restrict context to direct callers and callees. These functions are categorized as internal or external. As illustrated in Figure 2, the direct context of target function (node 3) includes two callers (nodes 1 and 2) and two callees (nodes 4 and 5), where nodes 1–4 are internal functions and node 5 is external.

We adopt distinct representation strategies for internal and external functions. For internal functions, we use function summaries as their representations. Although decompiled code is available, directly incorporating it presents two challenges: (i) its length may exceed the input limits of HIERONYM, and (ii) long caller or callee code can dilute attention and hinder the model’s focus on the target function. For example, in Figure 1, the decompiled code of the caller `png_read_row` spans 156 lines, whereas the target function contains only 30 lines. Function summaries provide a compact and effective alternative. First, summaries are less sensitive to instruction-level variations introduced by compilation, yielding robust representations. Second, compared with name-propagation approaches [54], which require iterative execution and are prone to error accumulation, summaries convey richer semantics in a concise form and enable parallel processing, improving both efficiency and stability. Finally, summaries reduce input length, better fit the context window of LLMs, and help the model focus on target function semantic. For external functions whose implementations are unavailable (e.g., dynamically linked library functions), we exploit the observation that their semantics can typically be inferred from their names (e.g., Linux system calls [22]). Accordingly, we directly use function names as their representations. The constructed local calling context comprises two components: summaries of internal functions and names of external functions. This design captures both internal implementation semantics and external calling interfaces, yielding a semantically rich and robust representation.

Target Function. We represent the target function using its decompiled code extracted from stripped binaries. Unlike raw assembly, decompiled code preserves richer structural and semantic information, making it more amenable to LLMs. Prior work has shown that using decompiled code as input can improve function renaming performance [35]. However, decompiler-generated code typically contains address-based function names that carry no semantic meaning. Directly incorporating such names can mislead model learning and degrade semantic understanding [35]. Therefore, we preserve the original code structure—including function bodies and parameters—while replacing address-based function names with a special [MASK] token. This strategy removes artificial naming noise and aligns the input representation with the masked-language modeling paradigm commonly used during LLM pretraining, thereby improving compatibility with the model’s learned representations.

4.4 Function Renaming Fine-Tuning

HIERONYM aims to achieve accurate and generalizable function renaming for stripped binaries. To this end, we fine-tune the LLM—already adapted to the binary domain—on a binary function renaming dataset, thereby optimizing its performance for the target task.

Parameter-Efficient Learning. Most existing binary function renaming models (e.g., SymLM [38] and BLens [4]) rely on full-parameter supervised fine-tuning, which updates all model parameters and incurs substantial computational cost. For example, SymLM requires up to 8 days of fine-tuning [38]. Given that HIERONYM is built upon a significantly larger LLM, full-parameter fine-tuning would be prohibitively expensive. To address this issue, we adopt a parameter-efficient fine-tuning strategy based on Low-Rank Adaptation (LoRA) [30]. LoRA freezes the original model parameters and introduces trainable low-rank matrices, dramatically reducing the number of trainable parameters while often achieving performance comparable to—or better than—full fine-tuning [30, 62].

HIERONYM is built on the LLM with a Transformer decoder architecture, whose core layers include masked multi-head attention, multi-head attention, additive normalization, and feed-forward layers. To efficiently adapt the model to binary semantics, we apply LoRA to the attention layers, which play a central role in semantic representation learning. Specifically, the original attention weight matrices are frozen to preserve the model’s pre-trained language understanding and instruction-following capabilities. During training, gradient updates ($\delta W \in \mathbb{R}^{d \times d}$) are instead captured through a low-rank decomposition into two trainable matrices $W_V \in \mathbb{R}^{d \times r}$ and $W_\Delta \in \mathbb{R}^{r \times d}$. This reduces trainable parameters from $d \times d$ to $2 \times r \times d$ ($r \ll d$), yielding substantial savings in computation and memory. The low-rank updates are incorporated into the original model via residual connections [28], preserving expressive capacity while enabling adaptation to the binary function renaming task.

Fine-Tuning Ground Truth and Objective. During the fine-tuning, we follow the common practice established in prior work [12, 20, 35, 38, 73] and use the original developer-defined function names as ground truth which are extracted from binaries with symbol tables. To ensure compatibility with the model’s output space, each ground-truth name is tokenized using the tokenizer associated with the base model¹.

We formulate binary function renaming as an autoregressive sequence generation task. The input to HIERONYM is denoted as $\mathcal{I}$, which consists of three components: global binary context $\mathcal{B}$, local calling context $\mathcal{C}$, and target function $\mathcal{T}$. The calling context $\mathcal{C}$ includes the callers ($\mathcal{C}^+$) and callees ($\mathcal{C}^-$) of $\mathcal{T}$, i.e., $\mathcal{C} = \{\mathcal{C}^+, \mathcal{C}^-\}$, and the input is $\mathcal{I} = \{\mathcal{B}, \mathcal{C}, \mathcal{T}\}$. Given a tokenized input sequence $\mathcal{I} = \{i_1, i_2, \dots, i_n\}$ and an output sequence $\mathcal{W} = \{w_1, w_2, \dots, w_m\}$, HIERONYM employs a forward autoregressive factorization and is trained to maximize the conditional log-likelihood:

$$\max_{\theta} \log P_{\theta}(\mathcal{W}) = \sum_{t=1}^m \log P_{\theta}(w_t \mid \mathcal{I}; w_{1:t-1}) \quad (2)$$

$$\log P_{\theta}(w_t \mid \mathcal{I}; w_{1:t-1}) = \frac{\exp(\mathbf{h}_{\theta}(\mathcal{I}; w_{1:t-1})^{\top} \mathbf{e}(w_t))}{\sum_{w' \in \mathcal{V}} \exp(\mathbf{h}_{\theta}(\mathcal{I}; w_{1:t-1})^{\top} \mathbf{e}(w'))} \quad (3)$$

where θ denotes the trainable parameters of HIERONYM, $w_{1:t-1}$ represents the sequence of tokens generated prior to predicting w_t , $\mathbf{h}_{\theta}(\cdot)$ is the hidden state vector produced by the intermediate layers of HIERONYM for contextual representation, $\mathbf{e}(\cdot)$ is the corresponding token embedding, and $\mathcal{V}$ is the vocabulary of HIERONYM.

¹We use the Hugging Face tokenizer provided with the base model of HIERONYM.

5 Evaluation

In this section, we conduct a comprehensive experimental evaluation to answer the following research questions:

RQ1: How well does HIERONYM rename stripped-binary functions?

RQ2: How does HIERONYM compare with SOTA methods?

RQ3: How does each component affect overall performance?

RQ4: How well does HIERONYM generalize?

5.1 Experimental Setup

Environment. All experiments are run on a server equipped with 48-core Intel Xeon Platinum 8558P CPU, 2 TB of RAM, and 8*NVIDIA H100 GPUs, running Ubuntu 22.04 with Python 3.10. We use Ghidra [16] (version 11.2.1) to extract functions from binaries and generate decompiled code. Although HIERONYM is decompiler-agnostic, Ghidra is selected due to its open-source availability and widespread adoption in prior research. To ensure fair comparison, we adopt Code Llama model as the base model, consistent with the SOTA method SymGen [35]. Code Llama is a code-specialized variant of Llama 2, pretrained on six programming languages [52]. Parsing of decompiled code is implemented using ANTLR 4 [46]. Parameter-efficient fine-tuning is conducted using PyTorch [48], the Hugging Face Transformers library [66], and LlamaFactory [76]. For function-name preprocessing and evaluation, we use NLTK [5], python-Levenshtein [25], and the Qwen3-Embedding model [75].

Baselines. In this paper, we select SymLM [38], XFL [50], SymGen [35], and BLens [4] as baselines for comparison. All baselines are trained on our dataset under the same experimental settings as HIERONYM to ensure a fair comparison, and we report results from the best-performing hyperparameter configurations following each method's training guidelines. XFL is applicable only to x64 binaries and BLens builds upon XFL's representations, both XFL and BLens are evaluated exclusively on the x64 dataset and compared against HIERONYM. We exclude NERO [12], NFRE [20], or Epitome [73] from comparison, as prior studies have shown that the selected baselines consistently outperform them. We also omit llasm [54], as its full source code is unavailable and its results could not be reproduced despite attempts to contact the authors.

5.2 Dataset Construction

We evaluate HIERONYM on the dataset introduced by SymGen [35], which comprises 33 open-source projects and their corresponding binaries compiled with GCC 9.4.0 across four optimization levels (O0–O3) and four architectures (x86, x64, ARM, and MIPS) (the detailed statistics are provided in Appendix D). The selected projects originate from widely used libraries, which are extensively employed in prior function renaming studies [20, 38, 73].

Function Renaming Dataset. The dataset contains 9,842 unique binaries and 2,237,915 binary functions in total. Unlike prior methods such as SymGen and SymLM, which train separate models for each architecture–optimization pair, HIERONYM trains a single model per architecture, thereby reducing training overhead. Following prior work [35, 38, 73], we randomly split the dataset into training, validation, and test sets in an 8:1:1 ratio.

To prevent data leakage [26], we split the dataset based on source code: binaries derived from the same source file but compiled with different optimization levels are assigned to the same sets. Although

function duplication due to code reuse and shared libraries is common in real-world binaries, we follow the stricter deduplication strategy of SymGen to evaluate generalization. Specifically, we remove functions that share the same name, have identical bodies despite different names, or originate from the same source but differ only due to address-dependent callees. Consistent with previous studies [4, 50], we exclude empty functions, functions with meaningless names, and functions automatically generated by Ghidra.

Since HIERONYM adopts the same base model and dataset configuration as SymGen, whose study demonstrated that the dataset was not leaked into the pretraining corpus of Code Llama [52], potential data-contamination concerns are mitigated, ensuring a fair comparison with SOTA approaches.

Hierarchical Domain Adaptation Dataset. Following the procedure described in Section 4.2, we construct the domain-adaptation dataset exclusively from the training set. Specifically, we first generate summaries for code snippets from decompiled code with symbol tables using a code-specialized LLM, and then align these summaries with the corresponding stripped decompiled code snippets.

5.3 RQ1: Overall Effectiveness

Our analysis begins with a comprehensive evaluation of HIERONYM's performance. Token-level results across four architectures and four optimization levels are reported in Table 1, while name-level results across the four architectures are summarized in Table 2. Overall, HIERONYM demonstrates strong performance, though clear variations emerge across different architectures. At the token level, HIERONYM achieves weighted-macro precision, recall, and F1-score of 0.4487, 0.4223, and 0.4339, respectively. When examined by architecture, HIERONYM performs best on x64 binaries, attaining an average precision of 0.5186, a recall of 0.5329, and an F1-score of 0.5256. Results on x86 binaries are slightly lower, with a precision of 0.5063, a recall of 0.4630, and an F1-score of 0.4832. In contrast, performance declines on ARM and MIPS binaries: HIERONYM attains a precision of 0.4258, recall of 0.4220, and F1-score of 0.4239 for ARM, and the corresponding values are 0.3440, 0.2713, and 0.3031 for MIPS. In the name-level evaluation, the performance trend is largely consistent with that observed at the token level, although minor deviations arise on the ARM architecture. Specifically, HIERONYM achieves the highest accuracy on x64 (0.4254), followed by ARM (0.3455) and x86 (0.3220), with performance dropping markedly on MIPS (0.1709). As name-level evaluation is conducted on a sampled subset of the test set, these results may be affected by the dataset scale. Nevertheless, the overall trends remain consistent with token-level findings, indicating stable model behavior across architectures.

The observed performance gap between x86/x64 and ARM/MIPS architectures is consistent with that of baselines under identical settings. For example, HIERONYM achieves approximately 1.24× higher performance on x64 than on ARM, comparable to the 1.38× improvement reported for SymGen. We attribute this disparity primarily to architecture-specific factors, particularly the greater complexity of application binary interfaces in ARM and MIPS. Their calling conventions and register usage patterns introduce additional challenges for function renaming. Moreover, empirical analysis shows that ARM binaries compiled from the same source often yield longer and more structurally complex decompiled functions than their x64

Table 1: Overall Token-Level Performance of HIERONYM and Our Baselines. Note that ↑ indicates the improvement of HIERONYM over the corresponding baseline.

Architecture	Model	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
		O0			O1			O2			O3		
x64	HIERONYM	0.5262	0.5620	0.5435	0.5373	0.5342	0.5357	0.4991	0.5125	0.5057	0.5117	0.5230	0.5173
	SymGen	0.4103	0.3933	0.4016	0.4180	0.3953	0.4064	0.4158	0.3750	0.3943	0.4044	0.4000	0.4022
	BLens	0.1416	0.1031	0.1193	0.1397	0.1092	0.1225	0.0926	0.0602	0.0730	0.1188	0.0755	0.0923
	XFL	0.1041	0.0788	0.0897	0.1712	0.1126	0.1359	0.1000	0.0721	0.0838	0.1122	0.0680	0.0847
	SymLM	0.0376	0.0854	0.0522	0.0979	0.0550	0.0705	0.0749	0.0462	0.0571	0.0922	0.0569	0.0704
	HIERONYM vs SymGen	(↑28.24%)	(↑42.90%)	(↑35.34%)	(↑28.54%)	(↑35.14%)	(↑31.83%)	(↑20.03%)	(↑36.67%)	(↑28.25%)	(↑26.53%)	(↑30.74%)	(↑28.61%)
	HIERONYM vs BLens	(↑271.60%)	(↑445.12%)	(↑355.59%)	(↑284.62%)	(↑389.19%)	(↑337.35%)	(↑438.96%)	(↑751.35%)	(↑592.75%)	(↑330.71%)	(↑592.68%)	(↑460.42%)
	HIERONYM vs XFL	(↑405.46%)	(↑613.23%)	(↑505.93%)	(↑213.85%)	(↑374.42%)	(↑294.22%)	(↑399.08%)	(↑610.84%)	(↑503.47%)	(↑356.04%)	(↑669.08%)	(↑510.70%)
x86	HIERONYM vs SymLM	(↑1299.44%)	(↑558.11%)	(↑941.22%)	(↑448.83%)	(↑871.27%)	(↑659.93%)	(↑566.32%)	(↑1009.34%)	(↑785.65%)	(↑454.97%)	(↑819.11%)	(↑634.75%)
	HIERONYM	0.5016	0.4085	0.4503	0.5057	0.4671	0.4856	0.4981	0.4700	0.4836	0.5200	0.5065	0.5131
	SymGen	0.3656	0.3243	0.3437	0.4049	0.3468	0.3736	0.3825	0.3436	0.3620	0.3981	0.3766	0.3871
	SymLM	0.1518	0.0660	0.0920	0.1385	0.0674	0.0907	0.1017	0.0736	0.0854	0.1320	0.0903	0.1073
	HIERONYM vs SymGen	(↑37.19%)	(↑25.96%)	(↑31.00%)	(↑24.89%)	(↑34.68%)	(↑29.98%)	(↑30.23%)	(↑36.78%)	(↑33.60%)	(↑30.61%)	(↑34.49%)	(↑32.56%)
	HIERONYM vs SymLM	(↑230.41%)	(↑518.90%)	(↑389.41%)	(↑265.12%)	(↑592.96%)	(↑435.40%)	(↑389.79%)	(↑538.54%)	(↑466.31%)	(↑293.91%)	(↑460.89%)	(↑378.22%)
	HIERONYM	0.3862	0.3835	0.3848	0.4584	0.4542	0.4563	0.4530	0.4489	0.4510	0.4058	0.4015	0.4036
	SymGen	0.3144	0.3054	0.3098	0.2807	0.2544	0.2669	0.3192	0.3022	0.3105	0.2879	0.2670	0.2771
ARM	SymLM	0.0587	0.0263	0.0363	0.0472	0.0349	0.0401	0.0645	0.0394	0.0489	0.0496	0.0213	0.0298
	HIERONYM vs SymGen	(↑22.84%)	(↑25.56%)	(↑24.22%)	(↑63.29%)	(↑78.54%)	(↑70.96%)	(↑41.92%)	(↑48.55%)	(↑45.24%)	(↑40.95%)	(↑50.37%)	(↑45.66%)
	HIERONYM vs SymLM	(↑557.93%)	(↑1358.06%)	(↑960.15%)	(↑871.12%)	(↑1201.46%)	(↑1037.85%)	(↑602.33%)	(↑1039.40%)	(↑822.20%)	(↑718.11%)	(↑1784.86%)	(↑1254.42%)
	HIERONYM	0.3543	0.2650	0.3032	0.3630	0.2891	0.3218	0.3684	0.2809	0.3188	0.2902	0.2502	0.2687
MIPS	SymGen	0.1881	0.2029	0.1952	0.1812	0.2074	0.1935	0.1822	0.1787	0.1804	0.1345	0.1560	0.1445
	SymLM	0.0198	0.0097	0.0130	0.0256	0.0095	0.0139	0.0185	0.0070	0.0102	0.0128	0.0046	0.0068
	HIERONYM vs SymGen	(↑88.38%)	(↑30.61%)	(↑55.34%)	(↑100.31%)	(↑39.39%)	(↑66.33%)	(↑102.21%)	(↑57.21%)	(↑76.71%)	(↑115.75%)	(↑60.40%)	(↑85.97%)
	HIERONYM vs SymLM	(↑1689.62%)	(↑2631.95%)	(↑2232.52%)	(↑1317.81%)	(↑2943.21%)	(↑2215.46%)	(↑1891.52%)	(↑3913.38%)	(↑3025.39%)	(↑2167.03%)	(↑5339.78%)	(↑3851.88%)

*XFL targets only x64 binaries by design [50], and BLens [4] builds upon XFL. Accordingly, both are evaluated solely on the x64 dataset.

Table 2: Name-Level Accuracy of HIERONYM and SymGen.

Model	Architecture			
	x64	x86	ARM	MIPS
HIERONYM	0.4254	0.3220	0.3455	0.1709
SymGen	0.3331	0.2251	0.1530	0.0765

counterparts, with a higher incidence of variable-type recovery errors. These architecture-dependent characteristics, combined with errors introduced during the decompilation process, exacerbate the difficulty of semantic modeling, thereby contributing to the reduced performance of HIERONYM on ARM and MIPS architectures.

In addition, we conduct a manual evaluation on 1,600 predictions sampled from the C experiments to assess the reliability of our evaluation framework. The results show that token-level semantic alignment achieves 89.8% agreement with human judgment, while name-level evaluation reaches 89.2%. Moreover, the name-level metric reduces GPT hallucinations by 11.28%. These findings demonstrate that our evaluation framework is both reliable and closely aligned with human judgment.

5.4 RQ2: Baseline Comparison

To further deepen our analysis, we compare HIERONYM with baselines on the same dataset. Token-level and name-level evaluation results are reported in Table 1 and Table 2, respectively.

Token-Level Comparison. Overall, HIERONYM substantially outperforms all baselines, achieving improvements of up to 396.37%, 688.01%, and 547.72% in precision, recall, and F1-score, respectively. Traditional ML-based baselines exhibit notably weaker token-level performance on unseen functions. Among the x64-only methods, BLens achieves an average precision of 0.1232, recall of 0.0870, and F1-score of 0.1018, while XFL reports comparable results (precision 0.1219, recall 0.0829, F1-score 0.0985). SymLM performs the weakest overall, with average precision, recall, and F1-score of 0.0702, 0.0433,

and 0.0515, respectively, across all architectures and optimization levels. In contrast, HIERONYM consistently surpasses these baselines by a wide margin. Compared with SymLM, HIERONYM achieves improvements of 860.27% in precision, 1598.83% in recall, and 1255.60% in F1-score. Although XFL outperforms SymLM on x64 binaries, HIERONYM still exceeds XFL by 343.61% in precision, 566.89% in recall, and 453.58% in F1-score. Similarly, relative to BLens, HIERONYM delivers achieves gains of 331.47%, 544.59%, and 436.52% in precision, recall, and F1-score, respectively. Notably, the performance of ML-based baselines on our dataset is substantially lower than that reported in their studies. For example, SymLM reports an average F1-score of 0.73 on its dataset but attains 0.0515 in our evaluation. This discrepancy is primarily attributable to data leakage in prior experimental setups, where duplicated functions between training and test sets inflate performance and obscure true generalization. Consistent with this explanation, SymLM reports F1-score below 0.10 on unseen binaries in its study [38]. Similar degradation on unseen functions has also been reported for XFL [50] and BLens [4]. Moreover, baseline methods that incorporate calling information are highly sensitive to optimizations: SymLM’s F1-score varies by 35.06%, XFL by 62.17%, and BLens by 67.81% across optimization levels. HIERONYM exhibits only a 7.48% variation, demonstrating strong robustness to optimization level changes. A similar trend appears across architectures. SymLM shows F1-score variation up to 753.64%, while HIERONYM exhibits substantially smaller variation (73.41%), highlighting robustness to architectural differences.

The LLM-based baseline SymGen achieves average token-level precision, recall, and F1-score of 0.3180, 0.3018, and 0.3093 across all architectures and optimization levels. Since we adopt the same configuration as in the original SymGen study [35], these results are not far from those previously reported. Even under this strong and directly comparable baseline, HIERONYM demonstrates a clear advantage, improving precision by 50.12%, recall by 41.75%, and F1-score by 45.10%. To further ensure fairness, we also re-evaluate

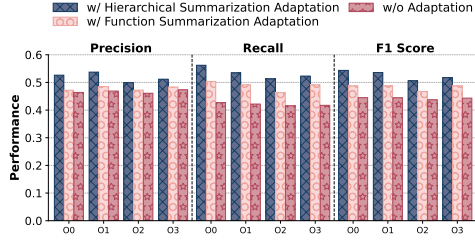


Figure 3: Effectiveness of Hierarchical Domain Adaptation

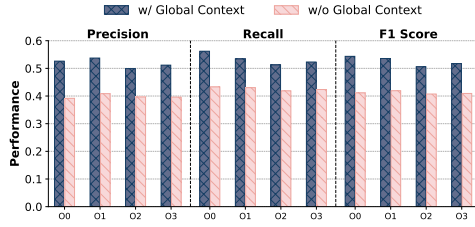


Figure 4: Effectiveness of Global Binary Context

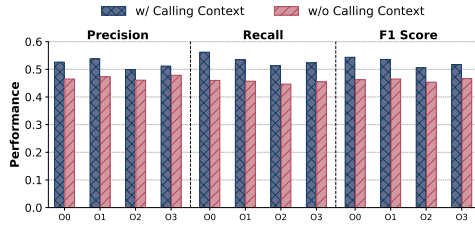


Figure 5: Effectiveness of Calling Context

performance using SymGen’s evaluation metrics, under which HIERONYM still improves F1-score by 47.96% over SymGen (see Appendix E). These results indicate that LLM-based methods generalize more effectively to unseen binaries than traditional ML methods, and that the multi-source information and hierarchical summarization-driven domain adaptation employed by HIERONYM further enhance semantic understanding and naming performance.

Name-Level Comparison. Due to budget constraints and the fact that ML-based methods are restricted to predicting labels from a closed vocabulary, we perform name-level evaluation only for HIERONYM and SymGen, as SymGen substantially outperforms other baselines at the token level. Under this setting, HIERONYM achieves an average accuracy improvement of 79.94% over SymGen across all architectures. This result further demonstrates that HIERONYM captures function semantics more comprehensively and generates function names with higher semantic accuracy.

5.5 RQ3: Ablation Study

In this section, we conduct ablation studies to obtain deeper insights into HIERONYM. Our goal is to systematically evaluate the contribution of each individual component to the overall performance. To

ensure a controlled and consistent comparison, all ablation experiments are performed on x64 dataset and use token-level metrics.

Effectiveness of Hierarchical Domain Adaptation. Domain adaptation is designed to enhance HIERONYM’s ability to capture binary code semantics by bridging the knowledge gap between the general-purpose pre-trained LLM and the binary domain. To evaluate the effectiveness of this strategy, we compare three model configurations: (1) hierarchical summarization-based domain adaptation, (2) function summarization only domain adaptation, and (3) no domain adaptation, where the model is directly fine-tuned on the function-renaming task using parameter-efficient methods. This comparison allows us to isolate and quantify the contribution of hierarchical summarization-based domain adaptation.

The results are shown in Figure 3. Across the four optimization levels (O0–O3) on the x64 architecture, hierarchical summarization-based domain adaptation improves HIERONYM over the no-adaptation baseline by an average of 11.14%, 26.76%, and 18.83% in precision, recall, and F1-score, respectively. Compared with an LLM adapted using function summarization only adaptation, HIERONYM yields average gains of 8.49%, 9.43%, and 8.95% on the same metrics. Function summarization only adaptation also outperforms the no-adaptation configuration, achieving average improvements of 2.44%, 15.85%, and 9.06% in precision, recall, and F1-score. These results lead to two key observations. First, domain adaptation enhances the LLM’s ability to understand binary code semantics. Second, incorporating hierarchical summarization provides additional benefits, demonstrating that fine-grained, statement-level semantic learning effectively complements function-level understanding and plays a important role in improving function renaming performance.

Effectiveness of Global Binary Context. To assess the impact of global binary context, we compare fine-tuned variants of HIERONYM with and without incorporating this context as input. As shown in Figure 4, incorporating global context yields a consistent performance gain, increasing the average F1-score from 0.4117 to 0.5256 (a relative improvement of 27.63%). This demonstrates the critical role of global context in function renaming, as it provides domain-relevant information that enhances the model’s ability to capture function semantics and generate accurate names.

To further verify that these gains are not driven by binary names that may appear in the global context, we remove test samples whose context contains such information and re-evaluate on the filtered set. HIERONYM achieves an F1-score of 0.444 on this subset, compared to 0.4339 on the original test set. The stable performance indicates that the improvements primarily arise from effective use of domain-relevant context rather than leakage from binary names.

Effectiveness of Calling Context. To evaluate the contribution of calling-context information, we compare two fine-tuned variants of HIERONYM: one that incorporates calling context in the input and another that excludes it. As shown in Figure 5, incorporating calling context consistently improves performance, yielding average gains of 10.52% in precision, 17.22% in recall, and 13.81% in F1-score. Notably, these improvements are more pronounced at lower optimization levels (O0/O1) and diminish at higher levels (O2/O3). We attribute this trend to the effects of aggressive compiler optimizations, such as function inlining, which remove explicit inter-function call relationships, thereby reducing the effectiveness of calling context. Overall, these results demonstrate that calling

Table 3: Cross-Compiler Performance

Architecture	HIERONYM			SymGen		
	Precision	Recall	F1-score	Precision	Recall	F1-score
x64	0.5841	0.6240	0.6034	0.4447	0.3950	0.4178
x86	0.5462	0.4916	0.5175	0.4253	0.3780	0.4001
ARM	0.3848	0.3442	0.3633	0.2708	0.2346	0.2512
MIPS	0.3626	0.3396	0.3507	0.1966	0.2038	0.2000

Table 4: Cross- Decompiler Performance

Architecture	HIERONYM			SymGen		
	Precision	Recall	F1-score	Precision	Recall	F1-score
x64	0.4817	0.4793	0.4805	0.4238	0.3394	0.3769
x86	0.4849	0.3994	0.4379	0.4451	0.3526	0.3931
ARM	0.5049	0.4427	0.4717	0.4167	0.3573	0.3846
MIPS	0.4753	0.4518	0.4632	0.4020	0.3736	0.3866

context provides valuable complementary semantic cues, enabling HIERONYM to construct richer function semantics representations and improve function renaming performance.

5.6 RQ4: Generalizability Test

In this section, we conduct a comprehensive evaluation of HIERONYM’s generalization across multiple dimensions, including cross-compiler, cross-decompiler, cross-project, and obfuscated settings. We use SymGen as the sole baseline, as it achieves the best performance in Section 5.4. To ensure fair comparison, all experiments adopt the same test set split and strict deduplication strategy as in Section 5.2, and are evaluated using token-level metrics.

Cross-Compiler Evaluation. We recompile the 33 projects in SymGen dataset using Clang 14.0.0 (details in Appendix D) and re-evaluate both HIERONYM and SymGen on the resulting binaries. As shown in Table 3, HIERONYM achieves a weighted macro F1-scores of 0.4587, compared to 0.3173 for SymGen, corresponding to a 49.68% improvement (see Appendix F for details). On the GCC-compiled test set, the F1-scores are 0.4339 for HIERONYM and 0.3093 for SymGen. The comparable performance across compilers indicates strong cross-compiler generalization for both methods, likely because LLM-based methods operate on decompiled code, which mitigates compiler-specific variation. Notably, HIERONYM further benefits from its design, leading to improved naming performance.

Cross- Decompiler Evaluation. We re-decompile the test set using IDA Pro 9.0 and re-evaluate both HIERONYM and SymGen on the resulting dataset. As shown in Table 4, HIERONYM achieves a weighted macro-average F1 score of 0.4633, compared to 0.3853 for SymGen, yielding a 20.54% improvement (see Appendix G for details). Compared to Ghidra, IDA can recover additional referenced string information—particularly for ARM and MIPS binaries—which can improve renaming performance. Consequently, both methods benefit from this change, with SymGen showing a more pronounced gain. Nevertheless, HIERONYM consistently outperforms SymGen, demonstrating robust cross-decompiler generalization.

Cross-Project Evaluation. We further deduplicate the test set by removing samples from projects present in the training set and re-evaluate on the resulting dataset. HIERONYM achieves a weighted macro-average precision of 0.4897, recall of 0.4461, and F1-score

Table 5: Performance on Obfuscated Binaries

Obfuscation	HIERONYM	SymGen
w/o obfuscation	0.4095	0.2947
bcbobf	0.3866 (-5.58%)	0.2759 (-6.37%)
cfbcbf	0.3759 (-8.20%)	0.2679 (-9.12%)
subobf	0.303 (-0.17%)	0.2911 (-1.22%)

of 0.4661, while SymGen attains 0.3453 precision, 0.3318 recall, and 0.3377 F1-score. These results are comparable to those on the original test set, indicating that both methods maintain strong performance on samples from previously unseen projects and exhibit robust cross-project generalization. This also confirms that our deduplication strategy is both strict and effective.

Obfuscated Binary Evaluation. To evaluate the obfuscation resistance performance of HIERONYM, we use SymGen’s obfuscated binary dataset, which contains 363 binaries from six GNU projects (findutils, coreutils, curl, less, putty, and bash). The binaries are obfuscated using three techniques: bogus control flow (bcbobf), control-flow flattening (cfbcbf), and instruction substitution (subobf). We also include the original binaries (w/o obfuscation) for comparison. As shown in Table 5, HIERONYM achieves F1-scores of 0.3866, 0.3759, and 0.303 across the three obfuscation settings, outperforming SymGen by up to 40.42%. Relative to original binaries, HIERONYM exhibits only modest degradation (e.g., 0.17% on subobf), which is lower than that of SymGen. These results demonstrate the superior obfuscation robustness of HIERONYM, likely due to its generalizable learning paradigm.

Cross-Language Evaluation. We evaluate the transferability of HIERONYM on C++ binaries and Java code. First, we assess HIERONYM on the Xalan benchmark (x86-64 architecture) from SPEC CPU2017 [11], which contains numerous functions with object-oriented C++ characteristics. HIERONYM achieves an F1-score of 0.4093, compared to 0.3696 for SymGen. Although both methods perform worse than on the original x64 dataset (0.5256 and 0.4011, respectively), HIERONYM still surpasses SymGen by 10.71%. Next, we randomly select four Java projects from AndroZooOpen [42] and apply R8 [13] to obfuscate identifier names, yielding more than 4,400 functions. We then predict only method names. SymGen achieves an F1-score of 0.2977, whereas HIERONYM attains 0.3779, representing a 26.93% improvement. Compared with C++, performance degrades more substantially on Java, likely due to the larger semantic or structural differences between Java and C-family binaries, which pose additional challenges for generalization. Nevertheless, HIERONYM consistently outperforms prior methods across languages, demonstrating strong cross-language transferability.

6 Illustration

6.1 Qualitative Evaluation

In our qualitative analysis, we examine function names predicted by HIERONYM and SymGen, sampled from the inference results. Table 6 reports both ground-truth and predictions, tokenized using our tokenization method and SymGen’s tokenization method, respectively. Tokenization is necessary for token-level evaluation.

Table 6: Qualitative Examples of Predictions Made by HIERONYM and SymGen. Tok. (Ours) denotes our tokenization, and Tok. (SymGen) denotes SymGen’s tokenization.

Ground Truth			HIERONYM Prediction			SymGen Prediction		
Raw	Tok. (Ours)	Tok. (SymGen)	Raw	Tok. (Ours)	Tok. (SymGen)	Raw	Tok. (Ours)	Tok. (SymGen)
jitterc_mangle collector_gettid	jitter,c,mangle collector,get, thread,id	jitter,c,mangle collector,get,t,id	jitter_mangle collector_get_thread_ id	jitter,mangle collector,get,thread,id	jitter,mangle collector,get,thread, id	mangle_name get_thread_id	mangle,name get,thread,id	mangle,name get,thread,id
ctf_type_encoding check_message	ctf,type,encoding check,message	ct,f,type,encode check,message	ctf_type_encoding validate_message	ctf,type,encoding validate,message	ct,f,type,encode validate,message	get_size nc_mvcur	get,size nc,mv,cur	get,size nc,mvc,ur
is_ascii_string	is,ascii,string	be,ascii,string	libgettextpo_is_ascii_ string	lib,get,text,po,is,ascii, string	lib,get,text,po,is, ascii,string	c_isascii_ string	c,is,ascii,string	c,isa,sc,ii,string
png_set_IHDR	png,set,ihdr	png,set,i,h,d,r	libtextstyle_png_set_ ihdr	lib,text,style,png,set, ihdr	lib,text,style,png, set,i,h,d,r	set_field_type	set,field,type	set,field,type
linearization_ string	linearization, string	linear,ization,string	sensor_reading_type_ code_linearization	sensor,reading,type, code.linearization	sensor,read,type, code.linear,zation	gnuplot_ scale_type	gnuplot,scale, type	gnuplot,scale, type
test_standard_ methods	test,standard, methods	t,est,standard,method	test_asn1_standard_ methods	test,asn,standard, methods	t,est,asn,standard, method	test_pkey_id_ order	test,pkey,id, order	t,est,pkey,id, order

Several insightful observations can be drawn from these examples from projects such as poke, binutils, gettext, libpng, freeipmi, and openssl. First, HIERONYM accurately infers function names that include project-specific tokens (e.g., `jitter_mangle`). As discussed earlier, although SymGen captures the function’s semantics, it fails to recover such identifiers (e.g., `jitter`), highlighting the effectiveness of our multi-source information fusion strategy. Second, HIERONYM correctly infers names (e.g., `ctf_type_encoding`) where SymGen fails to capture the correct semantics and produces erroneous predictions. Even when the predictions differ lexically from ground truth, HIERONYM often generates semantically equivalent tokens (e.g., `check`, `validate`), demonstrating a deeper understanding of function semantics. In contrast, SymGen frequently fails to grasp the underlying semantics, yielding less meaningful names.

We also observe a form of inference bias in HIERONYM’s predictions, where additional tokens appear that are absent from the ground truth. This phenomenon primarily stems from two sources. (i) Global binary context may include binary names or related library identifiers, which can be incorporated into predictions—for example, tokens such as `libgettextpo` and `libtextstyle` reflect domain cues present in the global context. (ii) Calling context may introduce keywords relevant to function behavior but not part of the canonical name. For instance, when predicting `linearization_string`, HIERONYM introduces tokens such as `sensor` and `type` derived from its calling context. Despite these extraneous tokens, the predicted names consistently capture the core functionality of the target functions, indicating that HIERONYM remains robust in modeling essential function semantics.

Based on the tokenized outputs, we further analyze the differences between our tokenization strategy and SymGen’s, as well as their impact on token-level evaluation results. SymGen often produces overly fine-grained and sometimes erroneous segmentations (e.g., tokenizing ‘test’ into {t, est}, ‘ctf’ into {ct, f}, or decomposing ‘isascii’ into {isa, sc, ii}). In contrast, our approach aims to partition function names into semantically meaningful tokens: ‘isascii’ is correctly tokenized as {is, ascii}, while common abbreviations such as ‘ctf’ and frequent words like ‘test’ are retained as single tokens to avoid unnecessary fragmentation. These differences have a direct and substantial impact on token-level evaluation. Improper tokenization disrupts semantic coherence of tokens and introduces bias into metric computation. For example, for the

function name ‘c_isascii_string’, the F1-score under SymGen’s tokenization is only 0.25, whereas it increases to 0.857 when using our tokenization strategy. This improvement highlights the effectiveness of our approach in preserving semantic integrity during evaluation. Furthermore, we also conduct name-level evaluation, which is inherently insensitive to tokenization and provides a robust evaluation of the performance of function-renaming methods.

6.2 Effectiveness on Real-world Binaries

To evaluate HIERONYM’s generalization in real-world security analysis, we apply it to the task of function renaming on malware executables. We collect ELF-format malware samples from VirusShare [60] and retain only those containing symbol tables to ensure reliable ground-truth function names. We select various types of x64 Linux malware, such as CDorked, camelot, and DnsAmp. All symbol tables are stripped prior to evaluation, forming an external test set.

Without additional fine-tuning, we directly apply HIERONYM trained on our primary dataset—composed mainly of GNU projects—to rename 2,478 malware functions. Evaluation follows token-level metrics described in Section 3.1, comparing predicted names against ground truth extracted from the original symbol tables. HIERONYM achieves a precision of 0.4855, a recall of 0.4897, and an F1-score of 0.4876. Notably, despite substantial differences in functional intent between benign open-source software and malware, performance on malware binaries is comparable to that observed on our evaluation dataset. These results indicate that HIERONYM retains robust semantic modeling and name inference capability when applied to previously unseen and domain-divergent binaries, highlighting its strong generalization potential in practical security analysis.

7 Limitation & Future Work

Base Model. To ensure fair experimental comparison, we adopt the same base model and experimental setup as the SOTA method SymGen [35]. However, the landscape of generative LLMs is evolving rapidly, with increasingly capable models continually emerging. As HIERONYM is model-agnostic by design, its performance could be further improved by integrating larger or more advanced base models. While our experiments already demonstrate the effectiveness of the proposed approach, investigating its integration with alternative base models represents a promising direction.

Statement Extraction Effectiveness. During dataset construction, we employ heuristic rules to extract code statements from functions. The lack of objective metrics for assessing statement quality introduces potential limitations and leaves room for further refinement. Despite this, our experimental results demonstrate that statement-level summarization effectively supports binary function renaming. Future work will focus on improving the statement extraction algorithm to further enhance performance.

Dataset Quality. Dataset quality is a critical factor in effective model training. For the function renaming task, we employ Ghidra to generate decompiled code as model input. Nevertheless, decompiler outputs inevitably contain inaccuracies due to the inherent challenges of decompilation, and discrepancies across different decompilers may introduce systematic bias. Although HIERONYM is designed to be decompiler-agnostic, its performance could benefit from higher-fidelity decompilation results. In addition, the quality of function names directly affects dataset validity. Many non-descriptive or semantically ambiguous names remain prevalent in real-world binaries. Fundamentally addressing this noise typically requires expert-driven manual curation, which is costly and time-consuming. Developing automated techniques to filter low-quality function names represents an important direction for future work.

For the domain-adaptation dataset, we employ Qwen3-Coder-30B to generate natural-language summaries of code snippets, which serve as supervision for training HIERONYM. The quality and stylistic consistency of these summaries may vary depending on the LLM used. Although proprietary models (e.g., ChatGPT) often produce higher-quality outputs, their cost renders them impractical for large-scale dataset construction. Prior work [35] has shown that summaries generated by open-source LLMs can still effectively support function renaming task. We therefore expect that further improving the consistency of code-snippet summaries could yield additional performance gains for HIERONYM. Exploring this direction constitutes a promising direction for future research.

Evaluator Selection. Using local LLMs similar to those in HIERONYM as evaluators may introduce bias. We therefore adopt GPT as a widely used external judge. We further evaluate Qwen3-Code-30B on 1,600 samples from the C experiments as an alternative evaluator. It achieves 76.31% agreement with human judgments, substantially lower than GPT’s 87.56%. Developing open-source evaluators that match GPT’s performance is an important direction.

Inference Efficiency and Cost. HIERONYM requires an average of 20.69 tokens and 0.0333s per function name, compared to 14.86 tokens and 0.0245s for SymGen. HIERONYM’s total cost comprises global-context summarization (0.0371s), calling-context summarization (0.0351s), and name generation (0.0333s), with the first two stages parallelizable. On large binaries (2,284 functions), HIERONYM achieves a throughput of 3.27 functions/s, corresponding to an end-to-end inference latency of 0.3056s per function.

8 Related Work

8.1 Function Renaming For Stripped Binaries

Early research on binary function renaming primarily relied on traditional ML techniques. For example, Debin [27] transforms binary code into an intermediate representation and constructs variable

dependency graphs, using Conditional Random Fields (CRFs) to infer function names. Similarly, Punstrip [49] generates probabilistic binary fingerprints from engineered features and applies a CRF-based model for name prediction. XFL [50] formulates function renaming as a multi-label classification problem that predicts sets of words. More recent works have framed function renaming as a neural machine translation task, translating binary code into natural-language function names. NERO [12] represents calling context using augmented control-flow graphs (CFGs) derived from assembly instructions, but largely ignores non-call instructions, which are unavailable for many functions. NFRE [20] proposes structure-sensitive embeddings of assembly instructions to generate function names, while AsmDepictor [39] employs a Transformer-based model to encode function semantics from assembly code. SymLM [38] leverages a pretrained assembly-language model to capture execution behavior, and Epitome [73] combines a pretrained assembly-language model with graph neural networks over CFGs to learn function semantics. BLens [4] adopts a different perspective by casting function renaming as an image-captioning-like task, aggregating multiple assembly-based embeddings into an ensemble representation aligned with the latent space of function names. Despite their methodological differences, these approaches share a fundamental limitation: function names are typically generated by selecting tokens from a closed training vocabulary, which restricts generalization to unseen or semantically novel names. Recently, LLMs have demonstrated strong generalization capabilities for function renaming. llasm [54] integrates a pretrained assembly encoder with a decoder-only LLM for name generation. SymGen [35] uses decompiled code as its input and introduces a function-summarization-based adaptation strategy to guide name generation. In this work, we discuss the limitations of existing methods (Section 2.2) and compare HIERONYM with four SOTA methods—SymLM, XFL, BLens, and SymGen—in Section 5.4.

In addition, with the rapid advancement of LLMs, researchers have explored their applications to other stripped binary analysis tasks, including decompilation [41, 67], variable name recovery [68, 70], and variable type recovery [17, 65, 68]. LLM-based approaches introduce a new paradigm for stripped binary analysis and continue to drive progress in the field. These efforts complement our work and further advance research on stripped binary analysis.

8.2 Source Code Summarization

Source code summarization aims to generate concise and human-readable natural-language descriptions for code snippets. Since around 2016, the majority of approaches in this area have been data-driven [34]. One of the earliest seminal works [34] introduced an attention-based encoder-decoder architecture inspired by neural machine translation. Building on this paradigm, CodeBERT [19], a Transformer-based model pre-trained on paired programming and natural-language data, demonstrated strong performance when fine-tuned for statement-level summarization. Subsequent research has focused on exploiting richer structural information from source code. Several studies adopted graph-based neural networks to encode AST structures [40, 74]. Huang et al. [33] further incorporated reinforcement learning over both code tokens and AST sequences

to improve statement-level summarization. With the recent success of LLMs in natural language processing, increasing attention has been directed toward leveraging them for source code summarization. Sun et al. [57] demonstrated that LLMs can generate high-quality code summaries using zero-shot prompting. Subsequent studies [1, 21] showed that few-shot prompting can significantly outperform supervised approaches. These results highlight the strong potential of LLMs to produce accurate and semantically rich code summaries. In this work, we leverage these advances to construct a domain-adaptation dataset that provides high-quality semantic supervision for binary code comprehension.

9 Conclusion

We propose HIERONYM, a novel approach that leverages the strong generalization capabilities of LLMs to rename functions from decompiled code in stripped binaries. To bridge the knowledge gap between general-purpose LLMs and the binary domain, we introduce a hierarchical summarization-driven domain adaptation strategy. HIERONYM further integrates multi-source information to construct comprehensive, task-relevant semantics of binary functions, thereby substantially improving function-renaming performance. In addition, we propose a dual-level evaluation framework that combines token-level and name-level metrics to enable systematic and reliable evaluation. Experimental results show that HIERONYM improves token-level precision, recall, and F1-score by 50.12%, 41.75%, and 45.10%, respectively, and achieves a 79.94% improvement in name-level accuracy over the SOTA method. Ablation studies confirm the effectiveness of key design components, and experiments on real-world malware binaries further demonstrate HIERONYM's practical utility in security analysis.

Acknowledgments

We are grateful to the anonymous reviewers for their valuable guidance and insightful comments. This paper is supported by the National Natural Science Foundation of China (Grant No. 62502467).

References

- [1] Toufique Ahmed and Premkumar Devanbu. 2022. Few-shot training LLMs for project-specific code-summarization. *arXiv:2207.04237 [cs.SE]* <https://arxiv.org/abs/2207.04237>
- [2] Simone Aonzo, Yufei Han, Alessandro Mantovani, and Davide Balzarotti. 2023. Humans vs. Machines in Malware Classification. In *32nd USENIX Security Symposium*. USENIX Association, 1145–1162. <https://www.usenix.org/conference/usenixsecurity23/presentation/aonzo>
- [3] Gregory V. Bard. 2007. Spelling-error tolerant, order-independent pass-phrases via the damerau-levenshtein string-edit distance metric. In *Proceedings of the Fifth Australasian Symposium on ACSW Frontiers - Volume 68*. Australian Computer Society, Inc., 117–124.
- [4] Tristan Benoi, Yunru Wang, Moritz Dannehl, and Johannes Kinder. 2025. BLens: contrastive captioning of binary functions using ensemble embedding. In *Proceedings of the 34th USENIX Conference on Security Symposium*. USENIX Association, Article 353, 20 pages.
- [5] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural language processing with Python: analyzing text with the natural language toolkit*. " O'Reilly Media, Inc."
- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*. Curran Associates Inc., Article 159, 25 pages.
- [7] Max Brunsfeld. 2026. Tree-sitter: An Incremental Parsing System for Programming Tools. Retrieved May 7, 2026 from <https://tree-sitter.github.io/tree-sitter/>
- [8] Mahinthan Chandramohan, Yinxing Xue, Zhengzi Xu, Yang Liu, Chia Yuan Cho, and Hee Beng Kuan Tan. 2016. BinGo: cross-architecture cross-OS binary search. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. Association for Computing Machinery, 678–689. doi:10.1145/2950290.2950350
- [9] Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. 2024. Universal Self-Consistency for Large Language Models. In *ICML 2024 Workshop on In-Context Learning*. <https://openreview.net/forum?id=LjsjHF7nAN>
- [10] Elliot J. Chikofsky and James H Cross. 1990. Reverse engineering and design recovery: A taxonomy. *IEEE software* 7, 1 (1990), 13–17.
- [11] SPEC CPU2017. 2026. Xalan-C++. https://archive.apache.org/dist/xml/xalan-c/binaries/Xalan-C_1_10_0-suse_80_AMD_64-gcc_32.tar.gz
- [12] Yaniv David, Uri Alon, and Eran Yahav. 2020. Neural reverse engineering of stripped binaries using augmented control flow graphs. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–28.
- [13] Android Developers. 2026. Enable app optimization with R8. <https://developer.android.com/topic/performance/app-optimization/enable-app-optimization>
- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [15] Steven H. H. Ding, Benjamin C. M. Fung, and Philippe Charland. 2019. Asm2Vec: Boosting Static Representation Robustness for Binary Clone Search against Code Obfuscation and Compiler Optimization. In *2019 IEEE Symposium on Security and Privacy*. 472–489. doi:10.1109/SP.2019.00003
- [16] NSA's Research Directorate. 2025. Ghidra. Retrieved December 19, 2025 from <https://ghidra-sre.org/>
- [17] Luke Dramko, Claire Le Goues, and Edward J Schwartz. 2026. Idioms: A Simple and Effective Framework for Turbo-Charging Local Neural Decompilation with Well-Defined Types. (2026).
- [18] Dror G Feitelson, Ayelet Mizrahi, Nofar Noy, Aviad Ben Shabat, Or Eliyahu, and Roy Sheffer. 2020. How developers choose names. *IEEE Transactions on Software Engineering* 48, 1 (2020), 37–52.
- [19] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
- [20] Han Gao, Shaoyin Cheng, Yinxing Xue, and Weiming Zhang. 2021. A Lightweight Framework for Function Name Reassignment Based on Large-Scale Stripped Binaries. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, 607–619. doi:10.1145/3460319.3464804
- [21] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large Language Models are Few-Shot Summarizers: Multi-Intent Comment Generation via In-Context Learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Association for Computing Machinery, Article 39, 13 pages. doi:10.1145/3597503.3608134
- [22] Gitiles. 2026. Linux system call table. <https://chromium.googlesource.com/chromiumos/docs/+master/constants/syscalls.md>
- [23] Emre Güler, Cornelius Aschermann, Ali Abbasi, and Thorsten Holz. 2019. AntiFuzz: Impeding Fuzzing Audits of Binary Executables. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 1931–1947. <https://www.usenix.org/conference/usenixsecurity19/presentation/guler>
- [24] Hanyang Guo, Xiangping Chen, Yuan Huang, Yanlin Wang, Xi Ding, Zibin Zheng, Xiaocong Zhou, and Hong-Ning Dai. 2023. Snippet Comment Generation Based on Code Context Expansion. *ACM Trans. Softw. Eng. Methodol.* 33, 1, Article 24 (Nov. 2023), 30 pages. doi:10.1145/3611664
- [25] Antti Haapala. 2026. Python-Levenshtein. Retrieved January 1, 2026 from <https://github.com/ztane/python-Levenshtein>
- [26] Awni Y. Hannun, Chuan Guo, and Laurens van der Maaten. 2021. Measuring Data Leakage in Machine-Learning Models with Fisher Information. *ArXiv abs/2102.11673* (2021). <https://api.semanticscholar.org/CorpusID:232013768>
- [27] Jingxuan He, Pesho Ivanov, Petar Tsankov, Veselin Raychev, and Martin Vechev. 2018. Debin: Predicting Debug Information in Stripped Binaries. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 1667–1680. doi:10.1145/3243734.3243866
- [28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. *arXiv:1512.03385 [cs.CV]* <https://arxiv.org/abs/1512.03385>
- [29] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Commun. ACM* 59, 5 (2016), 122–131.

- [30] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=nZeVKeeFYf9>
- [31] Yuan Huang, Hanyang Guo, Xi Ding, Junhui Shu, Xiangping Chen, Xiapu Luo, Zibin Zheng, and Xiaocong Zhou. 2023. A Comparative Study on Method Comment and Inline Comment. *ACM Trans. Softw. Eng. Methodol.* 32, 5, Article 126 (July 2023), 26 pages. doi:10.1145/3582570
- [32] Yuan Huang, Xinyu Hu, Nan Jia, Xiangping Chen, Yingfei Xiong, and Zibin Zheng. 2019. Learning code context information to predict comment locations. *IEEE Transactions on Reliability* 69, 1 (2019), 88–105.
- [33] Yuan Huang, Shaohao Huang, Huanchao Chen, Xiangping Chen, Zibin Zheng, Xiapu Luo, Nan Jia, Xinyu Hu, and Xiaocong Zhou. 2020. Towards automatically generating block comments for code snippets. *Information and Software Technology* 127 (2020), 106373. doi:10.1016/j.infsof.2020.106373
- [34] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2016. Summarizing Source Code using a Neural Attention Model. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2073–2083. doi:10.18653/v1/P16-1195
- [35] Linxi Jiang, Xin Jin, and Zhiqiang Lin. 2025. Beyond Classification: Inferring Function Names in Stripped Binaries via Domain Adapted LLMs. In *32nd Annual Network and Distributed System Security Symposium, San Diego, California, USA, February 24-28, 2025*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/beyond-classification-inferring-function-names-in-stripped-binaries-via-domain-adapted-llms/>
- [36] Yanjie Jiang, Hui Liu, and Lu Zhang. 2019. Semantic relation based expansion of abbreviations. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 131–141.
- [37] Xin Jin, Jonathan Larson, Weiwei Yang, and Zhiqiang Lin. 2023. Binary Code Summarization: Benchmarking ChatGPT/GPT-4 and Other Large Language Models. *ArXiv abs/2312.09601* (2023). <https://api.semanticscholar.org/CorpusID:266335888>
- [38] Xin Jin, Kexin Pei, Jun Yeon Won, and Zhiqiang Lin. 2022. SymLM: Predicting Function Names in Stripped Binaries via Context-Sensitive Execution-Aware Code Embeddings. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1631–1645.
- [39] Hyunjin Kim, Jinyeong Bak, Kyunghyun Cho, and Hyungjoon Koo. 2023. A Transformer-based Function Symbol Name Inference Model from an Assembly Language for Binary Reversing. In *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, 951–965. doi:10.1145/3579856.3582823
- [40] Alexander LeClair, Sakib Haque, Lingfei Wu, and Collin McMillan. 2020. Improved Code Summarization via a Graph Neural Network. In *Proceedings of the 28th International Conference on Program Comprehension*. Association for Computing Machinery, 184–195. doi:10.1145/3387904.3389268
- [41] Gangyang Li, Xiuwei Shang, Shaoyin Cheng, Junqi Zhang, Li Hu, Xu Zhu, Weiming Zhang, and Nenghai Yu. 2025. PseudoFix: Refactoring Distorted Structures in Decompiled C Pseudocode. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering*. 841–853. doi:10.1109/ASE63991.2025.00075
- [42] Pei Liu, Li Li, Yanjie Zhao, Xiaoyu Sun, and John Grundy. 2020. Androzoopen: Collecting large-scale open source android apps for the research community. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 548–552.
- [43] Alessandro Mantovani, Simone Aonzo, Yanick Fratantonio, and Davide Balzarotti. 2022. {RE-Mind}: a first look inside the mind of a reverse engineer. In *31st USENIX Security Symposium*. 2727–2745.
- [44] Marcus J. Min, Yangruibo Ding, Luca Buratti, Saurabh Pujar, Gail Kaiser, Suman Jana, and Baishakhi Ray. 2024. Beyond Accuracy: Evaluating Self-Consistency of Code Large Language Models with IdentityChain. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=caW7LdAALh>
- [45] OpenAI. 2024. GPT-4o-2024-08-06. <https://platform.openai.com/docs/models. Large multimodal language model>.
- [46] Terence Parr. 2013. The definitive ANTLR 4 reference. (2013).
- [47] Luca Pascarella, Magiel Bruntink, and Alberto Bacchelli. 2019. Classifying code comments in Java software systems. *Empirical Software Engineering* 24, 3 (2019), 1499–1537.
- [48] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [49] James Patrick-Evans, Lorenzo Cavallaro, and Johannes Kinder. 2020. Probabilistic Naming of Functions in Stripped Binaries. In *Annual Computer Security Applications Conference*. Association for Computing Machinery, 373–385. doi:10.1145/3427228.3427265
- [50] James Patrick-Evans, Moritz Dannehl, and Johannes Kinder. 2023. Xfl: Naming functions in binaries with extreme multi-label learning. In *2023 IEEE Symposium on Security and Privacy*. IEEE, 2375–2390.
- [51] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. <https://api.semanticscholar.org/CorpusID:160025533>
- [52] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. *arXiv:2308.12950 [cs.CL]* <https://arxiv.org/abs/2308.12950>
- [53] Giuseppe Scanniello, Michele Risi, Porfirio Tramontana, and Simone Romano. 2017. Fixing faults in c and java source code: Abbreviated vs. full-word identifier names. *ACM Transactions on Software Engineering and Methodology* 26, 2 (2017), 1–43.
- [54] Zihan Sha, Hao Wang, Zeyu Gao, Hui Shu, Bolun Zhang, Ziqing Wang, and Chao Zhang. 2025. llasm: Naming Functions in Binaries by Fusing Encoder-only and Decoder-only LLMs. *ACM Trans. Softw. Eng. Methodol.* 34, 4, Article 93 (April 2025), 22 pages. doi:10.1145/3702988
- [55] Temple F Smith, Michael S Waterman, et al. 1981. Identification of common molecular subsequences. *Journal of molecular biology* 147, 1 (1981), 195–197.
- [56] Zian Su, Xiangzhe Xu, Ziyang Huang, Kaiyuan Zhang, and Xiangyu Zhang. 2024. Source Code Foundation Models are Transferable Binary Analysis Knowledge Bases. *arXiv:2405.19581 [cs.SE]* <https://arxiv.org/abs/2405.19581>
- [57] Weisong Sun, Chunrong Fang, Yudu You, Yun Miao, Yi Liu, Yuekang Li, Gelei Deng, Shenghan Huang, Yuchen Chen, Quanjin Zhang, Hanwei Qian, Yang Liu, and Zhenyu Chen. 2023. Automatic Code Summarization via ChatGPT: How Far Are We? *arXiv:2305.12865 [cs.SE]* <https://arxiv.org/abs/2305.12865>
- [58] Qwen Team. 2025. Qwen3 Technical Report. *arXiv:2505.09388 [cs.CL]* <https://arxiv.org/abs/2505.09388>
- [59] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971 [cs.CL]* <https://arxiv.org/abs/2302.13971>
- [60] VirusShare BitTorrent Tracker. 2025. Virus-share. https://tracker.virusshare.com:7000/torrents/VirusShare_ELF_20200405.zip.torrent?C4D1472F2A6CF95D473246C6002E755C4B623603
- [61] Daniel Votipka, Seth Rabin, Kristopher Micinski, Jeffrey S. Foster, and Michelle L. Mazurek. 2020. An Observational Investigation of Reverse Engineers' Processes. In *29th USENIX Security Symposium*. USENIX Association, 1875–1892. <https://www.usenix.org/conference/usenixsecurity20/presentation/votipka-observational>
- [62] Tu Vu, Brian Lester, Noah Constant, Rami Al-Rfou, and Daniel Cer. 2022. SPoT: Better Frozen Model Adaptation through Soft Prompt Transfer. *arXiv:2110.07904 [cs.CL]* <https://arxiv.org/abs/2110.07904>
- [63] Shu Wang, Xinda Wang, Kun Sun, Sushil Jajodia, Haining Wang, and Qi Li. 2023. GraphSPD: Graph-Based Security Patch Detection with Enriched Code Semantics. In *2023 IEEE Symposium on Security and Privacy (SP)*. 2409–2426. doi:10.1109/SP46215.2023.10179479
- [64] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoko Okazaki (Eds.). Association for Computational Linguistics, 13484–13508. doi:10.18653/v1/2023.acl-long.754
- [65] Yanzhong Wang, Ruigang Liang, Yilin Li, Peiwei Hu, Kai Chen, and Bolun Zhang. 2025. Typeforge: Synthesizing and selecting best-fit composite data types for stripped binaries. In *2025 IEEE Symposium on Security and Privacy*. IEEE, 1–18.
- [66] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Mi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. HuggingFace's Transformers: State-of-the-art Natural Language Processing. *arXiv:1910.03771 [cs.CL]* <https://arxiv.org/abs/1910.03771>
- [67] Wai Kin Wong, Daoyuan Wu, Huaijin Wang, Zongjie Li, Zhibo Liu, Shuai Wang, Qiye Tang, Sen Nie, and Shi Wu. 2025. DecLLM: LLM-Augmented Recompileable Decompilation for Enabling Programmatic Use of Decompiled Code. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA081 (June 2025), 24 pages. doi:10.1145/3728958
- [68] Danning Xie, Zhuo Zhang, Nan Jiang, Xiangzhe Xu, Lin Tan, and Xiangyu Zhang. 2024. Resym: Harnessing llms to recover variable and data structure symbols from stripped binaries. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 4554–4568.
- [69] Xiangzhe Xu, Shiwei Feng, Yapeng Ye, Guangyu Shen, Zian Su, Siyuan Cheng, Guanhong Tao, Qingkai Shi, Zhuo Zhang, and Xiangyu Zhang. 2023. Improving Binary Code Similarity Transformer Models by Semantics-Driven Instruction

- Deemphasis. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, 1106–1118. doi:10.1145/3597926.3598121
- [70] Xiangzhe Xu, Zhuo Zhang, Zian Su, Ziyang Huang, Shiwei Feng, Yapeng Ye, Nan Jiang, Danning Xie, Siyuan Cheng, Lin Tan, et al. 2025. Unleashing the Power of Generative Model in Recovering Variable Names from Stripped Binary. In *Proceedings of the Network and Distributed System Security Symposium*.
- [71] Hiromu Yakura, Shinnosuke Shinozaki, Reon Nishimura, Yoshihiro Oyama, and Jun Sakuma. 2018. Malware Analysis of Imaged Binary Samples by Convolutional Neural Network with Attention Mechanism. In *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy*. Association for Computing Machinery, 127–134. doi:10.1145/3176258.3176335
- [72] Tab (Tianyi) Zhang, Claire Taylor, Bart Coppens, Waleed Mebane, Christian Collberg, and Bjorn De Sutter. 2025. reAnalyst: Scalable annotation of reverse engineering activities. *Journal of Systems and Software* 229 (2025), 112492. doi:10.1016/j.jss.2025.112492
- [73] Xiaoling Zhang, Zhengzi Xu, Shouguo Yang, Zhi Li, Zhiqiang Shi, and Limin Sun. 2024. Enhancing Function Name Prediction using Votes-Based Name Tokenization and Multi-task Learning. *Proc. ACM Softw. Eng.* 1, FSE, Article 75 (July 2024), 24 pages. doi:10.1145/3660782
- [74] Xiaoling Zhang, Shouguo Yang, Luqian Duan, Zhe Lang, Zhiqiang Shi, and Limin Sun. 2021. Transformer-XL With Graph Neural Network for Source Code Summarization. In *2021 IEEE International Conference on Systems, Man, and Cybernetics*. 3436–3441. doi:10.1109/SMC52423.2021.9658619
- [75] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).
- [76] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyuan Luo, Zhangchi Feng, and Yongqiang Ma. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics. <http://arxiv.org/abs/2403.13372>
- [77] Tehseen Zia and Usman Zahid. 2019. Long short-term memory recurrent neural network architectures for Urdu acoustic modeling. *Int. J. Speech Technol.* 22, 1 (March 2019), 21–30. doi:10.1007/s10772-018-09573-7

A Prompts for Global Context Summarization and Hierarchical Summary Generation

Prompt design plays a critical role in determining the quality of LLM outputs. To generate global binary information as well as function-level and statement-level summaries, we experimented with multiple prompt variants on a small-scale validation dataset and ultimately selected the prompt configurations that yielded the best empirical performance.

For generating task-related global binary context, we construct an input that includes a set of string literals extracted from the stripped binary, file-type metadata, and dynamic linkage dependencies. This information is provided to a LLM, which is instructed to produce a 100–150-word textual summary that captures the program's core functionality and overall application context. The corresponding prompt template is shown in Figure 6, and an example of the resulting global binary context is presented in Figure 7.

The prompt used for function-level summary generation is illustrated in Figure 8. In this setting, the original function name in the decompiled code is replaced with a special [MASK] token, while the function body, parameters, and other structural elements remain

Prompt: Global Binary Context Summarization

Role: You are an expert reverse engineering assistant specializing in binary analysis.
Task: Synthesize the provided static analysis data into a concise, semantic summary of a third-party library binary. Your summary will provide the critical context needed for the accurate semantic recovery of function names in a later phase of the analysis.

Summary Instructions:

- Content: Analyze the provided data to describe the program's core functionality and purpose. Your summary must be an inference-based synthesis, highlighting:
 - Core Capabilities: The principal algorithms, cryptographic routines, or protocols it implements.
 - Key Behavior: Its main operational logic, data handling patterns, and communication methods.
 - System Interaction: Significant interactions with the operating system, libraries, or hardware, and any standard error codes it uses.
- Format: Deliver the output as a single, self-contained paragraph in plain English. Aim for approximately 100–150 words.
- Constraints: Do not include code blocks, JSON, raw data dumps, compilation details, or any meta-commentary. Provide only the final, polished summary.

Input Data:

The static analysis information for the binary will be provided in the following format:

- Strings: {string_info}
- LDD: {ldd_str}
- File: {file_str}

Output:

Generate the final descriptive summary directly.

Figure 6: Prompt Template for Global Binary Context Summarization

Example: Global Binary Context

Output: "This binary is the wget2 command-line utility, a modern HTTP/FTP client designed for robust and efficient web content retrieval. Its core capabilities include handling HTTP/HTTPS requests with support for features like HTTP/2, TLS, cookies, authentication, and robots.txt parsing. It interacts extensively with system libraries such as libpcre2 for regex operations, libc for standard functions, and libwget for core networking and protocol handling. Key behaviors involve downloading files recursively, managing session cookies, processing MIME types, and supporting advanced options like HSTS, compression, and progress tracking. It communicates over TCP/IP, performs DNS resolution, and handles SSL/TLS connections, utilizing system-level error reporting and standard library calls for resource management and network I/O."

Figure 7: Example for Global Binary Context

Prompt: Function Summary Generation

Role: You are a helpful, respectful and honest assistant proficient in understanding programming languages. Your task is to generate detailed natural language descriptions for given function code.

Input You Will Receive: A section of C language code.

Your Tasks:

Generate Descriptions for ****Each Input Code Snippet****:

1. Describe what the code does from a functional perspective, highlighting the key inputs, outputs, main steps and the main purpose of function.
2. Describe the structural features that remain consistent after compilation.
3. Avoid unnecessary details and focus on delivering a high-level overview.

Guidelines for Generating Descriptions:

1. Provide Detailed Explanations:

- Avoid technical jargon unless it is explained.
 - Ensure that each description thoroughly explains the code's functionality and mechanisms.
 - Limit each description to a single paragraph of no more than 200 words.
2. Use Direct and Engaging Language:
- Start sentences with verbs.
 - Do not mention the function name which start with 'FUN_'

3. ****Output Format****: Present your answer as a JSON object with the following structure:
- ```
```json
{
  "code_descriptions": ""
}
```

For Example, you are provided with:

```
```C
void add_history_time (const char *string) {
 HIST_ENTRY *hs;
 if (string == 0 || history_length < 1)
 return;
 hs = the_history[history_length - 1];
 FREE (hs->timestamp);
 hs->timestamp = savestring (string);
}
```

Your output:

```
```python
{
  "code_description": "Adding or updating a timestamp for the most recent entry in a command history list. Before making any changes, it first verifies that the input time string is valid and that the history isn't empty—if either check fails, it exits immediately to prevent errors. Once confirmed, it locates the latest history entry, removes its old timestamp (freeing memory to avoid leaks), and replaces it with a fresh copy of the new timestamp string. The function produces no direct output but modifies the history entry in place. Its main purpose is to maintain accurate temporal metadata for command history entries, ensuring each recorded command can be associated with its execution time. The implementation relies on underlying utilities for string duplication and memory deallocation."
}
```


```

**Figure 8: Prompt for Function Summary Generation**

unchanged. This design encourages the model to infer function semantics solely from its implementation rather than relying on name cues. An example of a generated function-level summary is shown in Figure 9. Similarly, the prompt for statement-level summarization, shown in Figure 10, is specifically crafted to help HIERONYM capture fine-grained semantic information at the statement level. An example of a statement-level summary is provided in Figure 11. Together, these prompts enable the LLM to generate both high-level functional intent summary and low-level implementation logic summary. To further guide the model toward accurately understanding the task objectives, both function-level and statement-level prompts adopt a one-shot format, in which a complete input–output example is provided. This in-context demonstration constrains the LLM’s generation behavior and steers it toward producing summaries that better match the desired format and semantic granularity.

**Example: Function Summary**

**Output:** "Retrieving authentication credentials from a ticket object based on a provided URL and challenge string. The function accepts a ticket structure, URL, challenge message, and pointers to store either plain text credentials or secret objects. It validates input parameters and delegates credential retrieval to a callback function within the ticket structure. If the callback fails or is unavailable, the function falls back to interactive input handling, prompting the user with the challenge message and reading their response from standard input. The function manages terminal settings to disable echo when collecting sensitive information, ensuring secure input handling. It supports two modes of credential storage: plain text strings or secret objects, with appropriate memory allocation and reference counting. The function performs comprehensive error checking, including stack canary validation for security, and returns appropriate status codes indicating success or various failure conditions such as invalid parameters, memory allocation issues, or I/O errors. The implementation demonstrates robust error handling and secure credential management practices while maintaining compatibility with both programmatic and interactive credential acquisition methods."

**Figure 9: Example for Function Summary****Prompt: Statement Summary Generation**

**Role:** You are a helpful, respectful and honest assistant proficient in understanding programming languages. Your task is to generate detailed natural language descriptions for given code snippet.

**Input You Will Receive:** A section of C language code.

**Your Tasks:**

Generate Descriptions for **\*\*Each Input Code Snippet\*\***:

1. Describe what the code does from a functional perspective, highlighting the main steps and the main purpose of function.
2. Avoid unnecessary details and focus on delivering a high-level overview.

**Guidelines for Generating Descriptions:**

1. Provide Detailed Explanations:

- Avoid technical jargon unless it is explained.
  - Ensure that each description thoroughly explains the code's functionality and mechanisms.
  - Limit each description to one or two sentences.
2. Use Direct and Engaging Language:
- Start sentences with verbs.
  - Do not mention the function name which start with 'FUN\_'

3. **\*\*Output Format\*\***: Present your answer as a JSON object with the following structure: 

```
```json{  "code_descriptions": ""
}
```

For Example, you are provided with:

```
```C
if (regen_p (var) && var->dynamic_value){
 var = (*(var->dynamic_value)) (var);
 INVALIDATE_EXPORTSTR (var);
}
```
```

Your output must :

```
```json
{
 "code_descriptions": "Checks if a variable meets two specific conditions before performing operations on it. First, verifying whether the variable is eligible for regeneration using a function that evaluates its state. Second, checking if the variable contains a dynamically computed value. If both conditions are true, the code updates the variable by calling a function referenced in its dynamic value field, effectively regenerating its content. After updating the variable, the code marks any associated exported string as invalid."
}
```

**Figure 10: Prompt for Statement Summary Generation**

## B Prompt for Function Renaming Task Fine-Tuning

For fine-tuning the LLM on the function renaming task for stripped binaries, we employ the prompt template shown in Figure 12. The prompt input consists of three components: (i) global binary context; (ii) local calling context, where external callees are represented solely by their function context names, while internal callers and callees

**Example: Statement Summary**

**Output:** "Validating ticket credentials and handling authentication challenges by either retrieving existing values or prompting user input for plain text and secret information when necessary."

**Figure 11: Example for Statement Summary****Prompt: Function Renaming Task Fine-Tuning**

**Role:** You are a seasoned reverse engineering analyst specializing in reconstructing original symbols from decompiled binaries.

**Tasks:**

Analyze the provided information, you should infer code semantics and tell me the original function name from the contents of the function to replace [MASK].

**Output:** Provide only the function name. No explanations.

**Input:**

1. Environmental Context:  
{Global Binary Information}
2. Calling Context:  
2.1. names of external callees: {callee\_external}  
2.2. summaries of partial internal callees:  
{callee\_internal}  
2.3. summaries of partial callers:  
{caller}
3. Decompiled Code:  
```C  
{Decompiled Code}

Figure 12: Prompt for Function Renaming Task Fine-Tuning

are represented by their summaries (for efficiency, only the five most salient summaries are retained); and (iii) decompiled code of the target function, processed in the same manner as in function-level summarization, with the original function name replaced by a [MASK] token. Consistent with our summarization tasks, we evaluated multiple prompt templates on a small-scale dataset and ultimately selected the configuration that achieved the best empirical performance for formal fine-tuning.

C Prompts for Evaluation

To comprehensively evaluate the performance of function renaming methods, we adopt two complementary evaluation strategies: token-level evaluation and name-level evaluation.

For token-level evaluation, function names are tokenized into semantically meaningful tokens. We leverage the contextual reasoning capabilities of LLMs and employ few-shot prompting to guide the tokenization process, the corresponding prompt design is illustrated in Figure 13. This approach enforces segmentation only when semantic boundaries can be identified with high confidence, thereby reducing erroneous splits that would otherwise introduce noise into the evaluation.

For name-level evaluation, we exploit the discriminative capabilities of LLMs to determine whether a predicted function name is semantically consistent with the ground-truth name, given the binary's global context and the decompiled code of the target function. The prompt template used for this evaluation is shown in Figure 14. This strategy emphasizes holistic semantic alignment, is insensitive to incorrect tokenization, and more directly reflects the practical plausibility and usefulness of predicted function names in real-world analysis.

Prompt: Function Name Tokenization

Role: You are an expert code analysis assistant specialized in analyzing function names. Your task is to split function name into meaningful parts.

Input You Will Receive: function names.

Your Tasks:

Split name into meaningful parts while adhering to the following rules:

1. Identify and preserve: standard English words, common programming abbreviations, established technical terms or domain-specific terms.
2. Never split recognized technical terms or standard abbreviations.
3. Expand abbreviations only when you're absolutely certain of their meaning.
4. Prioritize conventions from reverse engineering and legacy code analysis.

Output Guidelines:

1. You should think step by step, the split function name is lowercase.
2. ****Output Format**:** Present your answer as a JSON object with the following structure:

```
```json
{
 "new_name": new_name
}
```

3. **\*\*No Code in Output\*\*:** Do not include any code in your output; only provide the JSON structure as specified.

4. **\*\*Direct Response\*\*:** Start your answer directly with the JSON output; do not include any explanations or additional text.

**Example:**

Given the following function name:

```
```C
Curl_os400_getsockname
```

Your output should be:

```
```json
{
 "new_name": curl_os_400_get_sock_name
}
```

Given the following function name:

```
```C
sockaddr2ascii
```

Your output should be:

```
```json
{
 "new_name": sock_address_to_ascii
}
```

Given the following function name:

```
```C
wctomb
```

Your output should be:

```
```json
{
 "new_name": wc_to_mb
}
```

Given the following function name:

```
```C
mbtowc
```

Your output should be:

```
```json
{
 "new_name": mb_to_wc
}
```

Given the following function name:

```
```C
msgid
```

Your output should be:

```
```json
{
 "new_name": msg_id
}
```

**Now, given the function name you receive, please provide the JSON output as specified according to the guidelines above.**

**Figure 13: Prompt for Function Name Tokenization**

**Table 7: Source Projects and Binaries in Our Dataset**

Project	# Binaries(GCC)	# Binaries(Clang)	Project	# Binaries(GCC)	# Binaries(Clang)
openssl-3.0.6	2128	2164	libconv-1.17	48	40
coreutils-8.32	1848	1736	grep-3.8	48	16
ncurses-6.3	1440	1168	gmp-6.2.1	40	60
readline-8.2	568	440	libidn2-2.3.4	32	28
bash-5.2	472	64	tar-1.34	32	32
adns-1.6.0	448	256	datamash-1.8	32	32
gettext-0.21	444	436	curl-7.86.0	32	28
inetutils-2.4	400	380	gss-1.0.4	32	28
binutils-2.39	352	452	poke-2.4	32	88
dico-2.11	276	216	libpng-1.6.39	32	148
gawk-5.2.1	240	216	gzip-1.12	16	16
libredwg-0.12	192	168	libmicrohttpd-0.9.75	16	16
mailutils-3.8	180	1288	libunistring-1.0	16	16
wget2-2.0.1	136	44	cflow-1.7	16	16
freipmi-1.6.10	112	280	libtool-2.4.7	16	16
diffutils-3.8	96	64	units-2.22	16	16
texinfo-7.0	54	32			

**Prompt: Function Name Similarity Quantification**

**Role:** You are a proficient software analysis assistant specializing in code semantics and binary analysis. Your task is to assess functional similarity between the given two function names.

**Task:**

Based on the given context, determine if Function A and Function B are \*functionally similar\* (i.e., perform equivalent or closely related operations).

- False = no functional similarity
- True = virtually identical functionality

**Constraints:**

- Do not guess; determine based on available evidence.
- Output must be "true" or "false" (NO quotes, brackets, spaces, or additional text).

**Output Format:**

boolean value

**Context:**

- Language: C
- Binary description: {bin\_info}
- Source code: {input}

**Input functions:**

- Function A: {groundTruth}
- Function B: {prediction}

**Table 8: Overall Performance of HIERONYM and SymGen Across Architectures(Arch) and Optimizations(Opt)**

Arch	Opt	HIERONYM			SymGen		
		Precision	Recall	F1-score	Precision	Recall	F1-score
x64	O0	0.4894	0.5262	0.5072	0.3547	0.3560	0.3553
	O1	0.4926	0.4932	0.4929	0.3602	0.3468	0.3534
	O2	0.4595	0.4745	0.4669	0.3683	0.3360	0.3514
	O3	0.4668	0.4808	0.4737	0.3582	0.3615	0.3599
x86	O0	0.4202	0.3435	0.3780	0.3152	0.2806	0.2969
	O1	0.4242	0.3940	0.4086	0.3408	0.2926	0.3149
	O2	0.4103	0.3900	0.3999	0.3167	0.2848	0.2999
	O3	0.4269	0.4177	0.4223	0.3313	0.3135	0.3222
ARM	O0	0.3364	0.3348	0.3356	0.2799	0.2729	0.2764
	O1	0.4003	0.3977	0.3990	0.2339	0.2129	0.2229
	O2	0.3962	0.3937	0.3949	0.2795	0.2648	0.2719
	O3	0.3507	0.3483	0.3495	0.2448	0.2273	0.2357
MIPS	O0	0.3327	0.2501	0.2856	0.1750	0.1893	0.1819
	O1	0.3376	0.2702	0.3002	0.1628	0.1865	0.1739
	O2	0.3514	0.2692	0.3048	0.1713	0.1681	0.1697
	O3	0.2730	0.2364	0.2534	0.1204	0.1397	0.1293

**Figure 14: Prompt for Function Name Similarity Quantification****D Source Project and Binaries in Our Dataset**

In our experiments, we adopt the SymGen dataset, which comprises 33 open-source projects and 9,842 unique binaries compiled with GCC 9.4.0 across four optimization levels (O0–O3) and four architectures (x86, x64, ARM, and MIPS). As shown in Table 7, the dataset includes representative GNU projects such as bash, wget, and coreutils, which are widely used in binary analysis research. To further evaluate the cross-compiler generalization of HIERONYM, we recompile these projects using Clang 14.0.0 across the same optimization levels and architectures, producing an additional 10,000 unique binaries.

**E Re-evaluation Based on SymGen Metrics**

To ensure fair comparison, we re-evaluate our results using the evaluation metrics adopted by SymGen. Specifically, SymGen splits

both ground-truth and predicted function names into tokens and computes standard precision, recall, and F1-score metrics. Both exact matches and semantically similar tokens are treated as correct, which is consistent with our token-level evaluation framework. To mitigate label noise caused by semantically related words, SymGen further employs CodeWordNet, introduced in SymLM [38], which leverages word embeddings to capture relationships among morphologically or semantically related tokens. In contrast, our approach adopts a multi-dimensional association mechanism to identify semantically equivalent tokens, improving robustness to lexical variability. Table 8 presents the experimental results across different architectures and optimization levels under SymGen’s evaluation setting. Even under this metric, HIERONYM consistently outperforms SymGen, achieving improvements of 53.23%, 44.33%, and 47.96% in precision, recall, and F1-score, respectively.

**Table 9: Cross-Compiler Performance of HIERONYM and SymGen Across Architectures(Arch) and Optimizations(Opt)**

Arch	Opt	HIERONYM			SymGen		
		Precision	Recall	F1-score	Precision	Recall	F1-score
x64	O0	0.5947	0.6362	0.6148	0.4044	0.3172	0.3556
	O1	0.5813	0.6087	0.5947	0.4803	0.4599	0.4699
	O2	0.5788	0.6204	0.5989	0.4621	0.4098	0.4344
	O3	0.5817	0.6308	0.6053	0.4318	0.3930	0.4115
x86	O0	0.5286	0.4726	0.4990	0.4377	0.3793	0.4064
	O1	0.5616	0.5078	0.5333	0.4207	0.3614	0.3888
	O2	0.5524	0.4984	0.5240	0.4192	0.3703	0.3932
	O3	0.5423	0.4879	0.5137	0.4236	0.4009	0.4119
ARM	O0	0.3876	0.3463	0.3658	0.2947	0.2792	0.2867
	O1	0.3890	0.3457	0.3661	0.2318	0.1894	0.2085
	O2	0.3820	0.3462	0.3632	0.2903	0.2496	0.2684
	O3	0.3805	0.3386	0.3583	0.2665	0.2202	0.2412
MIPS	O0	0.3692	0.3390	0.3534	0.2156	0.2269	0.2211
	O1	0.3601	0.3441	0.3519	0.1881	0.1922	0.1901
	O2	0.3584	0.3367	0.3472	0.2107	0.2028	0.2067
	O3	0.3629	0.3388	0.3505	0.1722	0.1931	0.1821

## F Cross-Compiler Generalization Results

To evaluate the cross-compiler generalization of HIERONYM, we re-evaluate both HIERONYM and SymGen on binaries compiled with Clang. To ensure fair comparison, the experiments adopt the same test split and strict deduplication strategy as Section 5.2 to prevent data leakage, and are evaluated using token-level metrics.

Table 9 summarizes the overall results across different architectures and optimization levels. HIERONYM achieves a weighted macro-average precision of 0.4694, recall of 0.4499, and F1-score of 0.4587, while SymGen attains 0.3344 precision, 0.3028 recall, and 0.3173 F1-score, corresponding to relative improvements of 47.46%, 52.19%, and 49.68%, respectively. On the original GCC-compiled test set, HIERONYM achieves precision, recall, and F1-score of 0.4487, 0.4223, and 0.4339, respectively, compared to 0.3180, 0.3018, and 0.3093 for SymGen. The comparable performance across compilers indicates strong cross-compiler generalization for both methods, likely because LLM-based approaches operate on decompiled code, which reduces compiler-specific variation. Notably, HIERONYM further benefits from its model design and consistently achieves superior naming performance. In addition, both methods exhibit architecture-dependent trends on the dataset compiled with Clang similar to those observed on the GCC-compiled dataset: performance remains strong on x64 and x86, but drops substantially on ARM and MIPS.

## G Cross-Decompiler Generalization Results

To evaluate the cross-decompiler generalization of HIERONYM, we re-decompile the test set using IDA Pro 9.0 and re-evaluate both HIERONYM and SymGen on the resulting dataset. The experiments also adopt the same strict deduplication strategy as Section 5.2 to prevent data leakage, and performance is evaluated using token-level metrics.

Table 10 summarizes the results across architectures and optimization levels. Overall, HIERONYM achieves a weighted macro-average precision of 0.4867, recall of 0.4433, and F1 of 0.4633, while SymGen attains 0.4219 precision, 0.3557 recall, and 0.3853 F1, corresponding to relative improvements of 15.56%, 25.21%, and 20.54%,

**Table 10: Cross-Decompiler Performance of HIERONYM and SymGen Across Architectures(Arch) and Optimizations(Opt)**

Arch	Opt	HIERONYM			SymGen		
		Precision	Recall	F1-score	Precision	Recall	F1-score
x64	O0	0.4758	0.4772	0.4765	0.4262	0.3514	0.3852
	O1	0.4808	0.4781	0.4794	0.4269	0.3388	0.3778
	O2	0.4899	0.4764	0.4830	0.4195	0.3280	0.3681
	O3	0.4803	0.4855	0.4829	0.4228	0.3392	0.3764
x86	O0	0.4867	0.4165	0.4489	0.4379	0.3716	0.4020
	O1	0.4753	0.3793	0.4219	0.4500	0.3419	0.3886
	O2	0.4851	0.3899	0.4323	0.4301	0.3108	0.3608
	O3	0.4926	0.4120	0.4487	0.4625	0.3861	0.4209
ARM	O0	0.5059	0.4332	0.4668	0.4419	0.3928	0.4159
	O1	0.5112	0.4495	0.4784	0.3961	0.3190	0.3534
	O2	0.5021	0.4439	0.4712	0.4149	0.3566	0.3835
	O3	0.5005	0.4440	0.4706	0.4141	0.3607	0.3856
MIPS	O0	0.4997	0.4673	0.4830	0.4164	0.3954	0.4056
	O1	0.4814	0.4586	0.4698	0.4056	0.3725	0.3858
	O2	0.4779	0.4481	0.4625	0.3984	0.3556	0.3758
	O3	0.4423	0.4333	0.4378	0.3875	0.3711	0.3791

respectively. For comparison, on the original GCC-compiled test set, HIERONYM achieves precision, recall, and F1 scores of 0.4487, 0.4223, and 0.4339, respectively, whereas SymGen achieves 0.3180, 0.3018, and 0.3093. These results indicate strong cross-decompiler generalization for both methods, with SymGen exhibiting particularly large gains. Further analysis shows that the improvement mainly arises from ARM and MIPS binaries. Manual inspection reveals that IDA Pro can often recover referenced string information directly within decompiled ARM functions, whereas Ghidra typically represents the same content as memory addresses. Such recovered strings provide valuable semantic cues for function name prediction. Because HIERONYM already incorporates global and call-context information in its design, which supplies additional semantic cues, the contribution of recovered strings is comparatively limited, although improvements on ARM and MIPS are still observed. In contrast, SymGen lacks such multi-source contextual inputs and therefore benefits more substantially from the additional string information. These findings suggest that improving decompilation quality is an effective direction for enhancing function renaming performance.